



The AREPO Public Code Release

Rainer Weinberger¹ , Volker Springel² , and Rüdiger Pakmor²

¹ Center for Astrophysics|Harvard & Smithsonian, 60 Garden Street, Cambridge, MA 02138, USA; rainer.weinberger@cfa.harvard.edu

² Max-Planck Institute for Astrophysics, Karl-Schwarzschild-Str. 1, D-85741 Garching, Germany
Received 2019 September 9; revised 2020 April 11; accepted 2020 May 4; published 2020 June 10

Abstract

We introduce the public version of the cosmological magnetohydrodynamical moving-mesh simulation code AREPO. This version contains a finite-volume magnetohydrodynamics algorithm on an unstructured, dynamic Voronoi tessellation coupled to a tree-particle-mesh algorithm for the Poisson equation either on a Newtonian or cosmologically expanding spacetime. Time integration is performed adopting local time step constraints for each cell individually, solving the fluxes only across active interfaces, and calculating gravitational forces only between active particles, using an operator-splitting approach. This allows simulations with high dynamic range to be performed efficiently. AREPO is a massively distributed-memory parallel code, using the message passing interface (MPI) communication standard and employing a dynamical workload and memory balancing scheme to allow for optimal use of multi-node parallel computers. The employed parallelization algorithms of AREPO are deterministic and produce binary-identical results when rerun on the same machine and with the same number of MPI ranks. A simple primordial cooling and star formation model is included as an example of sub-resolution models commonly used in simulations of galaxy formation. AREPO also contains a suite of computationally inexpensive test problems, ranging from idealized tests for automated code verification to scaled-down versions of cosmological galaxy formation simulations, and is extensively documented in order to assist adoption of the code by new scientific users.

Unified Astronomy Thesaurus concepts: Computational methods (1965); Computational astronomy (293); Magnetohydrodynamics (1964); Astrophysical fluid dynamics (101); Gravitation (661); Galaxy formation (595); Hydrodynamics (767); Large-scale structure of the universe (902)

1. Introduction

Computer simulations have become an essential part of modern astrophysical research (e.g., Naab et al. 2017). They allow for the numerical determination of the time evolution of initial-value problems, governed by (almost) arbitrarily complex partial differential equations. Over the past decades, computing power available for this approach has grown enormously, accelerating progress in the field by allowing ever more complex systems to be simulated with increasing levels of physical fidelity. However, continued rapid progress is only possible if advancements in computer hardware go hand in hand with the development of more sophisticated simulation software, as well as with improvements of the employed numerical methods (Springel 2010a; Lehner & Pretorius 2014; Teyssier 2015, for reviews of some of the techniques). The former involves efficient parallelization to achieve workload and memory balance on distributed-memory machines with a minimum amount of communication, something that can be quite nontrivial to achieve for tightly coupled problems. The latter involves the use of more accurate and flexible numerical schemes, and in particular the ability to focus the computational efforts on regions of particular interest.

Because the simulation codes themselves become increasingly complex, a third challenge emerges in the form of a need to better organize the improvement of software capabilities in (potentially large) collaborative teams (Portegies Zwart 2018). This calls for detailed documentation, modularity and portability of astrophysical simulation software, as well as for a continuous verification of individual parts contributed by scientific users with often very diverse technical backgrounds and development styles.

In this paper, we describe the public version of the AREPO code,³ a multipurpose gravitational and finite-volume magnetohydrodynamical (MHD) code for cosmic structure formation and more general astrophysical problems. AREPO was originally presented in Springel (2010b), and has subsequently been applied to a wide range of astrophysical problems. These include cosmological simulations of galaxy formation (see Vogelsberger et al. 2020, for a recent review) in large volumes such as Illustris (Genel et al. 2014; Vogelsberger et al. 2014) and IllustrisTNG (Marinacci et al. 2018b; Naiman et al. 2018; Nelson et al. 2018, 2019a; Pillepich et al. 2018, 2019; Springel et al. 2018), cosmological zoom simulations of galaxy formation such as Auriga (Grand et al. 2017), isolated galaxies (Jacob et al. 2018), spiral structure in galaxies (Smith et al. 2014), stratified box simulations modeling a part of a disk galaxy (Simpson et al. 2016), wind-tunnel-like setups (Sparre et al. 2019), turbulent boxes (Bauer & Springel 2012; Mocz et al. 2017), SNe Ia (Pakmor et al. 2013), binary stars in a common envelope phase (Ohlmann et al. 2016), tidal disruption events (Goicovic et al. 2019), protoplanetary and accretion disks (Muñoz et al. 2014; Fiacconi et al. 2018), and astrophysical jets (Bourne & Sijacki 2017; Weinberger et al. 2017).

These scientific applications have become possible thanks to a large number of people contributing to expanding and further refining AREPO since its initial description by Springel (2010b), and by making the code available upon request to a sizable number of people outside the group of direct scientific collaborators of the original AREPO developers. However, the code had not been made publicly available to the full

³ Homepage: www.arepo-code.org, repository: gitlab.mpcdf.mpg.de/vrs/arepo.

astronomical community thus far, an undesirable situation that we aim to address here. We share the view that making simulation codes publicly available constitutes good scientific practice in computational astrophysics, as it supports transparency and reproducibility, helps to identify bugs more quickly, and ultimately accelerates progress in the field by allowing scientists to build up more easily on previous work. Other codes in cosmological structure formation have led the way in this regard, such as GADGET (Springel et al. 2001), RAMSES (Teyssier 2002), ATHENA (Stone et al. 2008), ENZO (Bryan et al. 2014), CHANGA (Menon et al. 2015), and PHANTOM (Price et al. 2018), to name just a few.

The public release version of AREPO presented here corresponds to the state of the master development branch of the code as of 2017 November, albeit with somewhat reduced functionality with respect to special modules and features. This was done in order to provide a significantly simplified and completely documented code, both at a user and a developer level, in order to enable scientists to use and extend the code without direct support by the authors. This paper serves as an overview of the available functionality, and it summarizes a number of the updates done over the years since the initial exposition of the code in Springel (2010b). It can also be read as an introduction of AREPO to new users. To facilitate the latter, we connect used variables in equations with the corresponding code parameter names, where appropriate. In the interest of conciseness, we refer interested readers wherever applicable to the original development papers for in-depth discussions on why certain implementations were chosen over alternative methods.

This paper is structured as follows. In Section 2, the basic equations are presented. We discuss the solvers for gravitational interactions and MHD in Sections 3 and 4, respectively. In Section 5, we discuss the creation and dynamics of the computational mesh on which the MHD equations are discretized. Section 6 describes the implementation of additional source and sink terms such as radiative cooling as well as the coupling to sub-resolution models using star formation as an example. In Section 7, we present the time integration, and in Section 8, we present the dynamic workload and memory balancing scheme in AREPO. Section 9 covers other aspects of the code, such as the on-the-fly subhalo identification and initial conditions creation as well as important implementation aspects of the code such as input and output, memory management, and the use of external libraries. Finally, we discuss a number of examples and test cases in Section 10, present the code development and support strategy in Section 11, and give a summary in Section 12.

2. Equations

AREPO solves the equations of a collisionless particle component and of hydrodynamics on a uniformly expanding, flat Friedmann–Lemaître–Robertson–Walker spacetime. The space expansion is described by a single quantity, the scale factor $a(t)$, defined by the line element (see, e.g., Mo et al. 2010)

$$ds^2 = c^2 dt^2 - a^2(t)[dr^2 + r^2(d\vartheta^2 + \sin^2 \vartheta d\varphi^2)]. \quad (1)$$

Co-moving quantities are defined as follows:

$$\mathbf{x}_c = \mathbf{x}/a, \quad (2)$$

$$\mathbf{v}_c = \mathbf{v} - \dot{a} \mathbf{x}_c, \quad (3)$$

$$\rho_c = a^3 \rho, \quad (4)$$

$$p_c = a^3 p, \quad (5)$$

$$\Phi_c = a\Phi + \ddot{a}a^2 \mathbf{x}_c^2/2, \quad (6)$$

$$\mathbf{B}_c = a^2 \mathbf{B}, \quad (7)$$

where $\mathbf{x}$ and $\mathbf{v}$ are the “proper” position and velocity vectors, respectively, ρ is the density, p is the pressure, Φ is the gravitational potential, and $\mathbf{B}$ is the magnetic field strength. The variables with a subscript c denote the respective “co-moving” quantities, which are introduced to absorb the evolution of these quantities due to space expansion in full or in part, and thus to simplify the corresponding equations. To transform equations from proper to co-moving variables, it is important to observe that not only the coordinates but also the time and spatial derivatives change according to

$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial t} \Big|_{\mathbf{x}_c} = \frac{\partial f}{\partial t} \Big|_{\mathbf{x}} + \frac{\dot{a}}{a} \mathbf{x}_c \cdot \nabla_c f, \quad (8)$$

$$\nabla_c f = a \nabla f. \quad (9)$$

Note that a static spacetime can be easily recovered from the cosmological form of the equations by setting $a = 1$ and $\dot{a} = \ddot{a} = 0$. We therefore present the equations with co-moving variables in the following, dropping the subscript c for simplicity.

2.1. Cosmological Background Evolution

The evolution of the scale factor a is given by the Friedmann equation, assuming a Λ cold dark matter cosmology:

$$H = H_0 [\Omega_0 a^{-3} + (1 - \Omega_0 - \Omega_\Lambda) a^{-2} + \Omega_\Lambda]^{1/2}, \quad (10)$$

with $H = \dot{a}a^{-1}$ and where Ω_0 is the matter density in the universe relative to the critical density, and Ω_Λ represents the corresponding density parameter for the cosmological constant. Note that the contribution of radiation to the cosmic expansion history can be neglected for the redshifts of interest.

2.2. Gravitational Potential

The gravitational potential Φ in an expanding spacetime can be obtained from the Poisson equation in proper space, replacing the variables with the co-moving ones, and using the Friedmann equation to connect second derivatives of the scale factor to the mean density,

$$\nabla^2 \Phi = 4\pi G(\rho_{\text{total}} - \rho_{\text{mean}}), \quad (11)$$

with G being the gravitational constant, and ρ_{total} and ρ_{mean} are the total and mean density, respectively.

2.3. Collisionless Fluid

The collisionless component (i.e., dark matter and stars in galaxies) can be described by the Vlasov equation

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial t} + \frac{\partial f}{\partial \mathbf{v}} \frac{\partial \mathbf{v}}{\partial t} \quad (12)$$

$$= \frac{\partial f}{\partial t} + \frac{\partial f}{\partial \mathbf{x}} \frac{\mathbf{v}}{a} - \frac{\partial f}{\partial \mathbf{v}} \left(\frac{\nabla \Phi}{a^2} + \frac{\dot{a}}{a} \mathbf{v} \right) = 0. \quad (13)$$

Applying the method of characteristics yields simple equations of motion,

$$\dot{\mathbf{x}} = \mathbf{v}/a, \quad (14)$$

$$\dot{\mathbf{v}} = -\frac{\nabla\Phi}{a^2} - \frac{\dot{a}}{a}\mathbf{v}, \quad (15)$$

which can be used to integrate the trajectories of discrete particles that sample the initial phase-space distribution function.

2.4. Ideal Magnetohydrodynamics

The equations of ideal MHD can be written as (Pakmor & Springel 2013)

$$\frac{\partial\rho}{\partial t} + \frac{1}{a}\nabla \cdot (\rho\mathbf{v}) = 0, \quad (16)$$

$$\frac{\partial\rho\mathbf{w}}{\partial t} + \nabla \cdot \left(\rho\mathbf{v}\mathbf{v}^T + \mathbf{I}p_{\text{tot}} - \frac{\mathbf{B}\mathbf{B}^T}{a} \right) = -\frac{\rho}{a}\nabla\Phi, \quad (17)$$

$$\begin{aligned} \frac{\partial\mathcal{E}}{\partial t} + a\nabla \cdot \left[\mathbf{v}(E + p_{\text{tot}}) - \frac{1}{a}\mathbf{B}(\mathbf{v} \cdot \mathbf{B}) \right] \\ = \frac{\dot{a}}{2}\mathbf{B}^2 - \rho(\mathbf{v} \cdot \nabla\Phi) + a^2(\mathcal{H} - \Lambda), \end{aligned} \quad (18)$$

$$\frac{\partial\mathbf{B}}{\partial t} + \frac{1}{a}\nabla \cdot (\mathbf{B}\mathbf{v}^T - \mathbf{v}\mathbf{B}^T) = 0. \quad (19)$$

Note that the momentum and energy equations take a slightly different form compared to, e.g., those of Bryan et al. (2014). We prefer this formulation, however, as the use of time derivatives of the auxiliary variables

$$\mathbf{w} = a\mathbf{v}, \quad (20)$$

$$\mathcal{E} = a^2E, \quad (21)$$

reduces the number of MHD source terms to a single magnetic field term in the energy equation. The total energy and pressure is calculated as

$$E = \rho u_{\text{th}} + \frac{1}{2}\rho v^2 + \frac{\mathbf{B}^2}{2a}, \quad (22)$$

$$p_{\text{tot}} = (\gamma - 1)\rho u_{\text{th}} + \frac{\mathbf{B}^2}{2a}. \quad (23)$$

Note that possible external heating and cooling, described through $a^2(\mathcal{H} - \Lambda)$, appear as source terms in the energy Equation (18).

3. Modeling Gravitational Interactions via N -body Dynamics

The Vlasov Equation (13) describes the time evolution of the phase-space distribution function of a collisionless fluid. However, the high dimensionality of this equation makes direct discretization on a grid a computationally challenging task (Yoshikawa et al. 2013). Therefore, it is useful to sample the phase-space density f via discrete particles and solve the equations of motion of these simulation particles (Equations (14) and (15)). The key computational bottleneck is the computation of the acceleration $-\nabla\Phi$ of each individual particle because the potential Φ is generated from all other particles. This leads to, in its pure form, a computational effort scaling with N^2 , where N is the number of simulation particles. To avoid this unfavorable scaling behavior

with particle number, AREPO uses two different techniques. The first one is an oct-tree algorithm, which groups distant particles and calculates their collective contribution to the overall force, while the second method is a grid-based approach, where the gravitational force is calculated on a Cartesian grid via Fourier methods and then interpolated to each particle position. Each of these techniques has its advantages and disadvantages. Therefore, AREPO offers to use a combination of both techniques, combining the efficiency and implicit periodicity of a particle-mesh algorithm with the ability to cover a large dynamic range, which is an inherent strength of the tree algorithms.

3.1. Hierarchical Multipole Expansion with an Oct-tree Algorithm

The Barnes–Hut (Barnes & Hut 1986) oct-tree algorithm starts with an all-enclosing root node, which is then split up recursively into eight sub-nodes of equal volume, provided there are still particles contained in these volumes (see Figure 1). The version implemented in AREPO is similar to the one presented in Springel (2005); in particular, only monopole moments⁴ are calculated for each node, and the parallelization strategy for walking the tree is similar, too. We note, however, that the tree structure in AREPO is newly constructed for each local time step instead of using dynamic tree updates as in Springel et al. (2005b). This introduces additional computational cost, but avoids subtle correlations of the force accuracy with the time stepping hierarchy, and reduces force errors in the centers of halos in case the tree has not been reconstructed for a long time.

The tree construction starts with an enclosing cubical root node in which the particles are inserted sequentially one by one. Each time a particle ends up in an already occupied leaf node, i.e., a node with a particle in it and without further sub-nodes, the corresponding sub-nodes are created and the particles are distributed in the sub-nodes recursively until all leaf nodes contain at most one particle. If there are two particles with (almost) identical positions, this can lead to a very deep hierarchy. To avoid this, the ordinary hierarchy is limited to a maximum level (by default 30), at which point one of the two particles is assigned to a randomly selected neighboring sub-node.

For each tree node, multipole moments are computed recursively and then used to approximate the gravitational forces. To this end, the tree is walked for each particle, starting at the root node. If the multipole expansion of a visited node is considered sufficiently accurate, it is evaluated and added to the accumulated force. Otherwise, the node is opened and the daughter nodes are considered in turn. Following Springel (2005), a relative opening criterion is normally used where a node is opened if

$$\frac{Gm}{d^2} \frac{L_{\text{node}}^2}{d^2} > \alpha|\mathbf{a}|. \quad (24)$$

Here m is the mass of the node, d is the distance of the particle to the node's center of mass, $|\mathbf{a}|$ is the absolute value of the acceleration, estimated from the previous time step, and α is the code parameter to control the opening criterion. This limits the error from every individual particle-node interaction to a small

⁴ Dipole moments are implicitly included as well because the center of mass of the nodes is used as the expansion center.

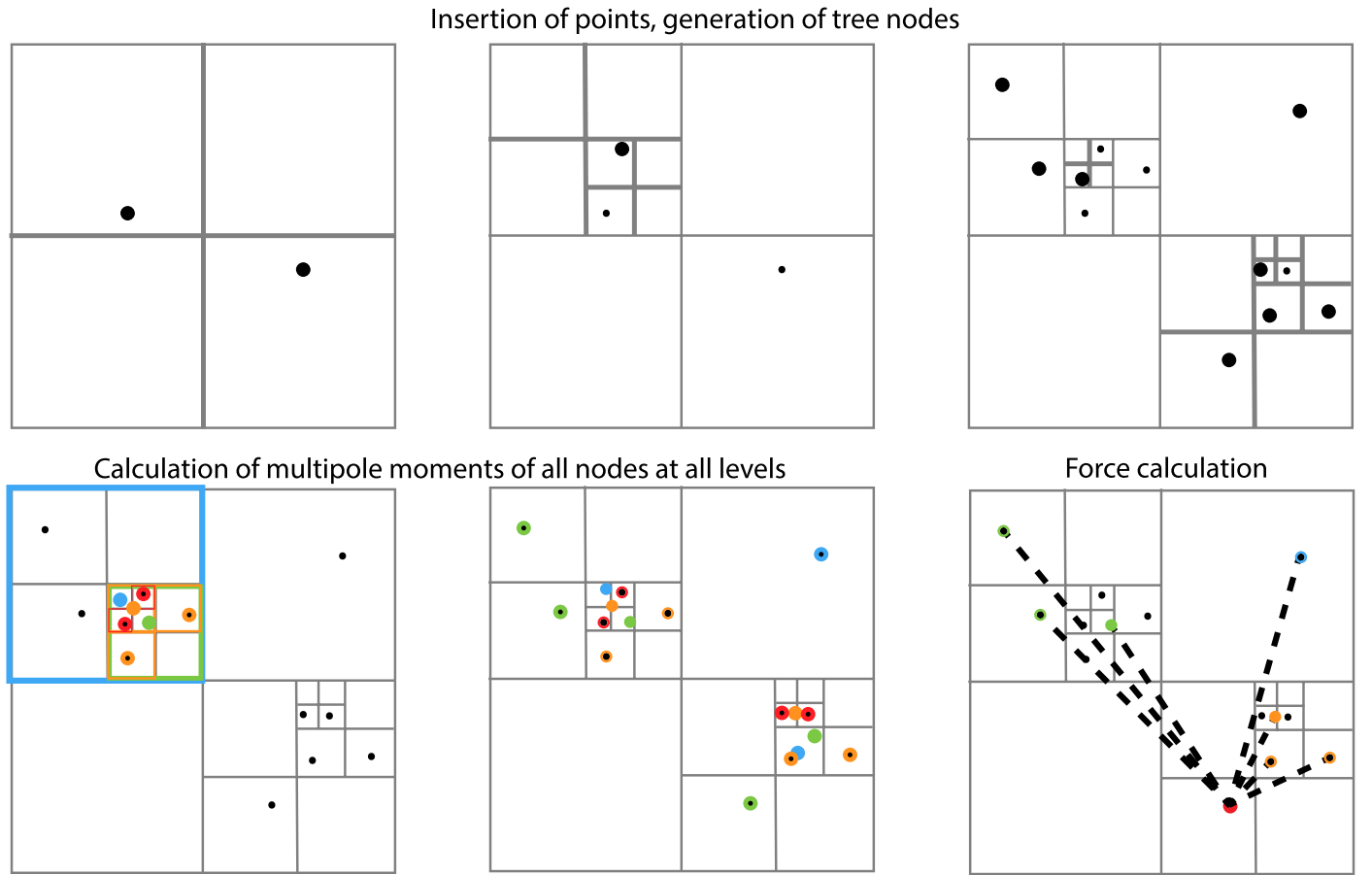


Figure 1. Illustration of tree construction, multipole-moment calculation, and force evaluation of one individual particle in an oct-tree algorithm (a quad-tree in the sketched 2D case). The upper panels illustrate the sequential insertion of simulation particles and the associated hierarchical splitting of the tree volume into sub-nodes. In the lower left and central panels, the calculation of multipole moments of all nodes is illustrated, with different colors denoting different depths of the tree hierarchy. Finally, the lower right panel illustrates the force evaluation between a single particle and the appropriate nodes. Reducing the number of these evaluations with respect to an evaluation with all simulation particles is the goal of this algorithm.

fraction of the total force, so that a roughly constant relative force error can be expected. In addition, a node is always opened if a particle lies within a box of size 1.2 times the node side length L_{node} , centered on the geometric center of the node. This guards against the possibility of unusually large force errors if a particle lies close to or within a node, but happens to be still relatively far away from the center of mass of the material in the box (Salmon et al. 1994).

A classic geometrical criterion

$$L_{\text{node}} > d \vartheta_{\text{opening}} \quad (25)$$

may also be used instead of the relative criterion, where the opening angle $\vartheta_{\text{opening}}$ is a parameter effectively controlling the resulting force accuracy. However, this criterion tends to be more costly at equal force accuracy than the relative criterion, making the latter the preferred choice. For the very first force calculation, the geometrical criterion is always used because no estimate of the absolute size of the force is available yet. If the relative criterion is in use, the force is then recomputed with the relative criterion.

Normally, the oct-tree algorithm takes into account the gravitational interactions of all particles in the simulation volume using a Newtonian force law, implying nonperiodic boundary conditions. If one, however, wants to compute the forces assuming periodic boundary conditions, the contributions

of (infinitely many) mirrored images need to be summed up as well. To achieve the latter, the Ewald summation technique for cosmological simulations presented in Hernquist et al. (1991) is used.

For isolated self-gravitating systems at rest with respect to the simulation coordinate system, the opening of nodes will be similar between different time steps. This means that the force errors made due to the approximation of the force via the tree are correlated at different times. Because all force errors in the tree algorithm do not necessarily add up to zero, this can lead to a net force on the system, and thus a nonconservation of momentum of the entire system that builds up with time.

To reduce this effect, AREPO randomizes the placement of the tree domain center for each tree construction for simulations with nonperiodic boundary conditions for gravity, which decorrelates the force errors in time and greatly improves the global momentum conservation. For simulations with periodic boundary conditions, AREPO instead shifts the whole box by a new random vector in every domain decomposition. This shift is done for all of the coordinate variables used in the code and is transparent to input and output.

3.2. Gravitational Softening

When sampling continuous phase-space distribution functions with discrete particles, it is important to realize that the

direct interaction between close particles would introduce artificial two-body interactions that violate the premise of collisionless dynamics. To avoid this, a gravitational force softening is introduced in N -body simulations of collisionless systems, which replaces the Newtonian force below a softening scale ϵ with a reduced force that smoothly declines to zero for vanishing distance. AREPO employs the same softening law as the GADGET code, where the potential of a point mass at zero lag is $-Gm/\epsilon$. The force becomes fully Newtonian at 2.8ϵ , but strong reductions of the force due to softening only occur for $d \sim \epsilon$ and smaller distances. The force softening de facto also limits the maximum acceleration one particle can cause, simplifying the orbit integration, and it can protect against the formation of bound particle pairs. A detailed analysis of the optimum softening value in the context of cosmic structure formation is presented in Power et al. (2003).

AREPO allows collisionless particles to have different softening lengths, either assigned through a concept of “particle types,” where the softening of each type can be set separately, or through properties of individual particles, such as the particle mass. For gas cells, the gravitational softening length is always chosen in a variable way,

$$\epsilon_{\text{cell}} = f_h \left(\frac{3V}{4\pi} \right)^{1/3}, \quad (26)$$

where V is the volume of the Voronoi cell and f_h is an input parameter that controls the size of the softening in relation to the cell size.

Interactions between particles with different softening lengths are symmetrized by adopting the larger of the two. If a node contains particles with different softening lengths, and the target particle’s softening is smaller than the node’s maximum softening length, and the distance to the node’s center of mass is smaller than the node’s maximum softening, a node is always opened, because otherwise the multipole expansion may effectively account for some interactions with the wrong symmetrized softening. Otherwise, the node may be used if permitted by the opening criteria, using a multipole expansion of the softened interaction potential where appropriate.

3.3. Particle-mesh Algorithm

Alternatively to calculating individual particle interactions, it is also possible to bin the mass distribution on a Cartesian, regular grid via cloud-in-cell (CIC) assignment, solve for the gravitational potential on this grid, finite difference it to get the force, and then interpolate the force field to the particle positions. A conceptionally simple way of doing this in the periodic case is to use a discrete Fourier transformation to convert the Poisson Equation (11) to

$$-k^2 \Phi_k = 4\pi G \rho_k. \quad (27)$$

Hence a simple division of the Fourier-transformed density field ρ_k with the square of the wavevector k is sufficient to obtain the Fourier-transformed gravitational potential. Using an inverse Fourier transform then yields the potential field, which subsequently can be differenced and interpolated to the particle position. In practice, we apply an additional deconvolution operator to correct for the smoothing effects of the CIC assignment and the tri-linear interpolation (Hockney & Eastwood 1981).

AREPO allows for the use of a mesh covering the entire computational domain to calculate large-scale forces (PMGRID⁵ option). This is commonly applied for cosmological volume simulations, where this approach also conveniently yields periodic boundary conditions. However, the code can also compute mesh-based forces for nonperiodic boundaries. In this case, zero-padding and a different Green’s function in Fourier space is used. For zoom simulations, it is additionally possible to place a second (nonperiodic) mesh onto the usually small high-resolution region in order to extend the dynamic range covered with this technique (PLACEHIGHRESREGION). For these types of simulations, an alternative communication algorithm for binning the density field and interpolating the forces (PM_ZOOM_OPTIMIZED) is also available, allowing reasonable workload balance in the PM calculations even for highly inhomogeneous particle distributions. The particle-mesh algorithm makes use of the one-dimensional fast Fourier transformation of the FFTW library. The use of external libraries is discussed in more detail in Section 9.5.

3.4. Tree-particle-mesh Approach

Both the tree and the particle-mesh methods have advantages and disadvantages. The particle-mesh algorithm is conceptionally simple, fast, and comparably easy to scale to a large number of message passing interface (MPI) tasks. However, it is severely limited by being bound to a uniform Cartesian mesh, limiting the dynamic range of the calculation significantly. This poses a severe obstacle for cosmological simulations of galaxy formation. On the other hand, the particle-mesh algorithm yields periodic boundary conditions in a computationally highly efficient and accurate way, which is perfectly suited for cosmological simulations.

The tree algorithm in its standard form assumes nonperiodic boundary conditions, and extending this to periodic boundaries is computationally costly, especially for the small density perturbations found on an otherwise homogenous background at high redshift. On the other hand, it can naturally and efficiently handle large dynamic ranges in spatial scales, allowing very high spatial force resolutions. In addition, it can be easily combined with local and adaptive time stepping. One disadvantage however is that scaling to a large number of MPI tasks is more difficult, especially if the particle distribution is highly clustered.

To benefit from the advantages of both methods, AREPO is able to split forces into short-range and long-range components and compute the former with its tree algorithm and the latter using the particle-mesh approach. The resulting TreePM algorithm (Bagla 2002) is implemented in a conceptually similar way as in GADGET (Springel et al. 2005b). The force is split into a long-range and a short-range contribution, where the short-range force of a point mass is obtained from the Newtonian force by multiplying by

$$f_l = 1 - \operatorname{erfc}\left(\frac{r}{2r_s}\right) - \frac{r}{\sqrt{\pi}r_s} \exp\left(-\frac{r^2}{4r_s^2}\right). \quad (28)$$

⁵ Here and in the following, we point out compile-time options of the code that are controlling features discussed in the text, for easier reference.

Table 1
Code Parameters for Gravitational Force Calculation

Description	Symbol	Code Parameter
Tree opening angle	$\vartheta_{\text{opening}}$	ErrTolTheta
Acceleration opening criterion	α	ErrTolForceAcc
Gas softening length relative to cell radius	f_h	GasSoftFactor
Force split scale	a_s	ASMTH
Short-range force cutoff	a_{cut}	RCUT

In Fourier space, the complementary long-range force is obtained from the long-range potential

$$\Phi_{k,\text{long}} = \Phi_k \exp(-k^2 r_s^2) \quad (29)$$

where the split scale r_s is specified in terms of a factor a_s , giving this length in units of the PM mesh cell size, i.e.,

$$r_s = a_s \frac{L_{\text{box}}}{N_{\text{pm}}}. \quad (30)$$

When walking the tree, nodes or particles farther away than a cutoff radius $a_{\text{cut}} r_s$ are simply ignored and need not be evaluated, because f_i has dropped to a negligible value there and the corresponding force is provided by the PM algorithm. The ability to discard the mass distribution except for a local neighborhood then accelerates the tree force calculation significantly, making TreePM ultimately a fast, accurate, and still very flexible force calculation approach.

For cosmological simulations, the code allows us to have two layers of particle-mesh calculations, as described above, the first covering the complete box, the second only the high-resolution region. The tree walk is then able to use a shorter cutoff length in the high-resolution region than in the more coarsely sampled boundary region of low resolution. The standard parallel fast Fourier transform (FFT) package used by AREPO supports parallel Fourier transforms with a slab-based decomposition. In this case, the mesh size directly limits the number of MPI ranks that can be efficiently used, which can become a restriction for very large simulations. To avoid this limitation, AREPO offers alternatively a column-based FFT (FFT_COLUMN_BASED). The corresponding algorithm requires more transpose operations and is therefore more costly for small transforms, but its better scalability ultimately allows larger maximum transform sizes. In Table 1, we summarize the names of code parameters affecting the gravity calculation, and give their mapping to the symbols used in the above equations.

4. Discretization of MHD

To solve the equations of (magneto)hydrodynamics, AREPO uses a second-order accurate finite-volume discretization. To this end, volume-averaged primitive variables ρ , $\mathbf{v}$, and $\mathbf{B}$ are stored as properties of the cell at its center. Gradients are estimated with the corresponding values for neighboring cells (Pakmor et al. 2016b), allowing for a piecewise-linear reconstruction of the solution. Using the gradients, the primitive variables are extrapolated to all mesh interfaces, for which fluxes are calculated by solving a Riemann problem locally at each interface. The flux calculation can be done either with an exact, iterative Riemann solver (default) or the approximate HLLC (RIEMANN_HLLC) solver in the case of

pure hydrodynamics and an HLLD (RIEMANN_HLLD) solver in the case of MHD (see Toro 1997, for details).

4.1. Gradient Estimate

As shown in Pakmor et al. (2016b), the gradient estimate of the hydrodynamic quantities originally presented in Springel (2010b), based on a finite difference formula for Voronoi cells, can become inaccurate for highly distorted cells. The present version of AREPO therefore employs an improved, least-square gradient estimate (Pakmor et al. 2016b). The basic idea is that the gradient ∇W_i of a primitive variable W_i is determined such that the linearly extrapolated values from cell i to the positions of the neighboring cells j ,

$$\tilde{W}_j = W_i + \mathbf{d}_{ij} \cdot \nabla W_i, \quad (31)$$

agree with the actual values W_j found there as well as possible. Here $\mathbf{d}_{ij}$ is the position vector of cell j relative to cell i . Because there are multiple neighbors, exact equality can generally not be achieved; rather, we determine a gradient estimate by minimizing the residuals of the over-determined set of equations in a weighted least-square sense. Specifically, we minimize

$$S_{\text{tot}} = \sum_j g_j (W_j - W_i - \mathbf{d}_{ij} \cdot \nabla W_i)^2. \quad (32)$$

The adopted weights are given by $g_j = A_{ij}/|\mathbf{d}_{ij}|$, where A_{ij} is the area of the interface between i and j .

4.2. Divergence Constraint

In their analytical form, the equations of MHD automatically conserve the $\nabla \cdot \mathbf{B} = 0$ constraint provided that it is fulfilled initially. This property, however, is generally not true for discretizations of these equations (see however Evans & Hawley 1988). In this version of AREPO, we adopt the divergence-cleaning method introduced by Powell et al. (1999), as implemented into the code by Pakmor & Springel (2013). This method advects numerically induced divergences away, and has the advantage of using cell-centered magnetic fields. Thus, the momentum, energy, and induction Equations (17)–(19) become

$$\frac{\partial \rho \mathbf{w}}{\partial t} + \nabla \cdot \left(\rho \mathbf{u} \mathbf{u}^T + p_{\text{tot}} \mathbf{I} - \frac{\mathbf{B} \mathbf{B}^T}{a} \right) = -\frac{1}{a} (\nabla \cdot \mathbf{B}) \mathbf{B}, \quad (33)$$

$$\begin{aligned} \frac{d\mathcal{E}}{dt} + a \nabla \cdot \left[\mathbf{u} (E + p_{\text{tot}}) - \frac{1}{a} \mathbf{B} (\mathbf{u} \cdot \mathbf{B}) \right] \\ = \frac{a}{2} \mathbf{B}^2 - (\nabla \cdot \mathbf{B}) (\mathbf{u} \cdot \mathbf{B}), \end{aligned} \quad (34)$$

$$\frac{\partial \mathbf{B}}{\partial t} + \frac{1}{a} \nabla \cdot (\mathbf{B} \mathbf{u}^T - \mathbf{u} \mathbf{B}^T) = -\frac{1}{a} (\nabla \cdot \mathbf{B}) \mathbf{u}. \quad (35)$$

The divergence of the magnetic field in a cell i is calculated as

$$\nabla \cdot \mathbf{B}_i = V_i^{-1} \sum_{\text{faces}} \mathbf{B}_{\text{face}} \cdot \hat{\mathbf{n}} A_{\text{face}}, \quad (36)$$

with $\hat{\mathbf{n}}$ being the normal vector to the cell, $\mathbf{B}_{\text{face}}$ is the value of the magnetic field on the interface, and V_i is the volume of the cell. The advantage of this method is that it is very flexible in terms of mesh geometry, and the possibility to discretize it on local time steps makes it computationally inexpensive and thus

suitable for use in cosmological simulations of galaxy formation. But the eight-wave formulation has known shortcomings, e.g., in reproducing jumps in an inclined 2D MHD shocktube test (Tóth 2000; Mignone & Tzeferacos 2010), and AREPO does not reach as low numerical reconnection rates as state-of-the-art Cartesian grid MHD codes (see magnetic current-sheet example, Section 10.6).

4.3. Gravitational Interactions of the Fluid

To compute the gravitational forces of the fluid, the gas cells are included in the gravitational force calculation as described above. The code uses cell-centered softened gravitational forces with softening lengths tied to the cell radii, such that the gravitational field created by the gas distribution is a good approximation of a space-filling continuous density distribution.

The gravitational forces are coupled as source terms to the MHD equations, in an operator split approach. This means the source terms are applied for half a time step at the beginning of a step, then the hyperbolic set of fluid equations are evolved for one time step in their conservative form, followed by another gravity half-step. In the case of a self-gravitating fluid, this approach does not manifestly conserve total energy, which is difficult albeit not impossible to achieve for mesh codes (Jiang et al. 2013). However, errors in total energy quickly decrease for better resolution, and the source term approach can avoid unphysical drifts in thermal energy in poorly resolved flows for conservative treatments (see also the discussion in Section 5 of Springel 2010b); hence, we prefer it in practice. For simplicity, we refrain from including correction factors in the forces due to changes in the gravitational softening lengths. They were still included in Springel (2010b) but are typically quite small in practical applications. And as they can act as an additional source of noise, it is unclear whether they are ultimately beneficial for the final accuracy.

5. Computational Mesh

A characteristic feature of the AREPO code is its discretization on a fully adaptive, dynamic Voronoi mesh. The starting point for creating this Voronoi mesh is mesh-generating points, each one representing the position of a Voronoi cell. Each Voronoi cell is defined as the region in space that is closer to a given mesh-generating point than to any other mesh-generating point. This implies that mesh interfaces are normal to the connection of two neighboring cells, and lie at equal distances to the mesh-generating points on either side.

The ensemble of connections to neighboring cells (and only to them) forms a so-called Delaunay triangulation, consisting of tetrahedra in three dimensions. The circumsphere of each tetrahedron does not contain any other mesh-generating point, and it is this mathematical property that makes the Delaunay tessellation special and unique among all other possible triangulations of space. The Delaunay tessellation is computed iteratively in AREPO: we start with four points (in three dimensions) that span a domain-enclosing tetrahedron. Now, we successively insert the mesh-generating points into the Delaunay mesh, adding connections from the point to the corners of the tetrahedron the point falls in. This splits the tetrahedron into four smaller tetrahedra. However, each of the resulting tetrahedra is not necessarily a Delaunay tetrahedron, i.e., it may violate the empty circum-circle property. Delaunayhood is, in this case, restored by iteratively

applying edge-flipping to affected tetrahedra (see Springel 2010b, for a detailed discussion of all possible cases) before the next point is inserted. Once the Delaunay triangulation is completed, all mesh-generating points are part of the Delaunay mesh, and the Voronoi mesh is trivially obtained as its topological dual, requiring only simple geometrical calculations such as the areas of the interfaces.

In simulations with distributed-memory parallelization, the above algorithm is first used to compute local meshes for each domain, which are then complemented at the borders by inserting further mesh-generating points imported as “ghost points” from neighboring domains. In this way, Voronoi cells that overlap with domain boundaries are still constructed with their correct shape. An illustration of the mesh in a single domain, and its connections to points from neighboring domains, is shown in Figure 2. How these domains are determined is discussed in Section 8.

5.1. Mesh Movement

Each mesh-generating point normally moves with the bulk velocity of the fluid in its cell, also taking into account accelerations due to local pressure gradients, the Lorentz force, and gravity, i.e., de facto adopting the expected velocity at the next half time step. This naturally leads to an approximate equal-mass discretization and to quasi-Lagrangian behavior. This, however, does not ensure by itself that the mesh remains reasonably regular at all times: due to local fluid motions, high aspect ratios of mesh cells can occur, which increase discretization errors and mesh noise, and may thus negatively impact the accuracy of the solution (as discussed in Springel 2010b; Duffell & MacFadyen 2011; Vogelsberger et al. 2012).

To avoid this, velocity corrections can be applied for highly distorted cells to steer the mesh motion (REGULARIZE_MESH_CM_DRIFT). We identify highly distorted cells by the maximum angle under which any of its faces is seen from the mesh-generating point (REGULARIZE_MESH_FACE_ANGLE), i.e.,

$$\alpha_{\text{face}} = (A_{\text{face}}/\pi)^{1/2}/h_{\text{face}}, \quad (37)$$

$$\alpha_{\text{max}} = \max(\alpha_{\text{face}}), \quad (38)$$

where A_{face} is the area of a given interface and h_{face} is the distance from the mesh-generating point to the interface. If α_{max} exceeds a predefined threshold 0.75β , with β being a free parameter, the mesh-generating point is moved with a fraction f_{shaping} of the characteristic speed in the cell toward its center of mass. The corrective velocity is parameterized as:

$$\mathbf{v}_{\text{corr}} = \begin{cases} 0 & \text{for } \alpha_{\text{max}} \leq 0.75\beta \\ f_{\text{shaping}} \frac{\alpha_{\text{max}} - 0.75\beta}{0.25\beta} v_{\text{char}} \hat{\mathbf{n}} & \text{for } 0.75\beta < \alpha_{\text{max}} \leq \beta \\ f_{\text{shaping}} v_{\text{char}} \hat{\mathbf{n}} & \text{for } \alpha_{\text{max}} > \beta \end{cases} \quad (39)$$

Vogelsberger et al. (2012, Section 2.2.2 (i)) discusses the use of this mesh regularization approach in the context of cosmological simulations. It has become the default for these kinds of simulations since. However, the modified Lloyd scheme presented in Springel (2010b) can also be used as an alternative (REGULARIZE_MESH_FACE_ANGLE not set), for which the

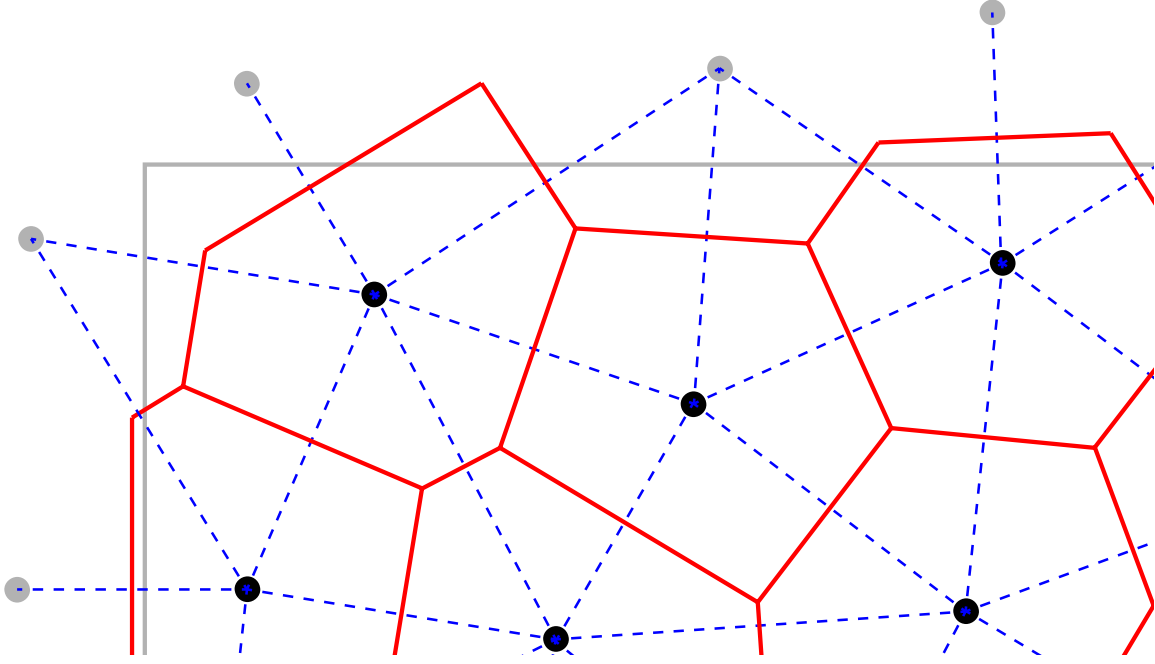


Figure 2. Schematics of the Voronoi mesh in a single domain and its connections to neighboring domains. The domain boundary is denoted in gray, the Delaunay connections in blue, and the Voronoi mesh in red. The mesh-generating points of the local domain are shown in black, and the ones of different domains, imported as ghost cells, are drawn as gray dots. Which domain a given cell is associated with is determined by the location of the mesh-generating point, not by the cell boundary. Unlike in a fixed grid-geometry, ghost points from other domains (or boundaries) need to be imported to determine the local mesh.

correction velocity is given by

$$v_{\text{corr}} = \begin{cases} 0 & \text{for } d \leq 0.75 \eta r_{\text{cell}} \\ f_{\text{shaping}} \frac{d - 0.75 \eta r_{\text{cell}}}{0.25 \eta r_{\text{cell}}} v_{\text{char}} \hat{n} & \text{for } 0.75 \eta r_{\text{cell}} < d \leq \eta r_{\text{cell}} \\ f_{\text{shaping}} v_{\text{char}} \hat{n} & \text{for } d > \eta r_{\text{cell}} \end{cases} \quad (40)$$

with d being the distance between the mesh-generating point and center of mass of a cell. For the characteristic speed v_{char} of a cell, the sound speed can be used (REGULARIZE_MESH_CM_DRIFT_USE_SOUNDSPEED), or $d/\Delta t$ where Δt is the time step. In the latter case, v_{char} is at most equal to the maximum hydrodynamic velocity in the system. Table 2 summarizes the most important run-time parameter of the code affecting the hydrodynamics and the mesh motion.

5.2. Refinement and De-refinement

Even when using quasi-Lagrangian mesh motion, over the course of many simulation time steps, cells can diverge from their desired mass content or intended size. For this reason, AREPO offers the possibility to refine and de-refine the mesh locally, thus offering full spatial adaptivity.⁶

To refine a cell, the mesh-generating point is split into a pair of two very close points, offsetting their location in a random direction by $0.025 r_{\text{cell}}$, where

$$r_{\text{cell}} = \left(\frac{3V}{4\pi} \right)^{1/3}. \quad (41)$$

⁶ Note that the refinement/de-refinement operations are local to a given cell and do not trigger any global load-balancing operations.

Table 2
Code Parameters for Hydrodynamics and Mesh Movement

Description	Symbol	Code Parameter
Adiabatic index	γ	GAMMA
Cell roundness criterion	β	CellMaxAngleFactor
Alternative cell roundness criterion	η	CellShapingFactor
Cell deformability parameter	f_{shaping}	CellShapingSpeed

This effectively splits the original Voronoi cell into two cells, without affecting the geometry of the neighboring cells. The conserved quantities of the split cell are then subdivided conservatively among the two new cells according to their volume ratio.

For de-refinement, a mesh-generating point is taken out, thereby removing the corresponding Voronoi cell from the mesh, with the neighboring cells claiming its volume. The conserved quantities associated with the eliminated cell are conservatively distributed among these neighbors, in proportion to the volume overlap they realize with the removed cell.

In principle, nearly arbitrary refinement criteria are possible, and very different ones have been employed in certain simulations (e.g., van de Voort et al. 2019). In the public version, we include an often used mass criterion, which triggers de-refinement if a cell has less than half of a desired target mass resolution, and refinement if it has more than twice this target mass. This keeps the mass resolution in a narrow corridor around the target mass resolution, thus realizing a Lagrangian behavior akin to smoothed particle hydrodynamics (SPH) codes. Additionally, a refinement criterion based on resolving the Jeans length by a given number of cells can be employed. This is usually used to ensure that gravitational collapse and fragmentation are resolved sufficiently well, and not significantly impacted by resolution issues.

We note that normally highly distorted cells are excluded from refinement, in order to avoid that refinements triggered in quick succession in subsequent time steps produce locally a very irregular mesh. After a refinement event, the mesh steering motions take a couple of time steps to reestablish a locally regular mesh geometry, at which point the next refinement may then proceed.

5.3. Boundary Conditions

For gravity, AREPO supports either periodic or nonperiodic (vacuum) boundary conditions. For hydrodynamics, periodic boundary conditions are one of the primary possibilities and represent the simplest choice, but the code also supports reflective and in-/outflow boundary conditions at the box borders. Differently from SPH and pure particle-based gravity, the confines of the simulated volume need to be unambiguously specified in AREPO. At least for hydrodynamics, a box size must therefore always be specified.

Reflective boundaries are effectively realized by mirroring the mesh-generating point set at the box border, thereby ensuring that Voronoi faces align with the box boundary. The fluid state is then also mirrored, so that the Riemann solvers return zero mass flux at the corresponding box border. Inflow or outflow boundaries are realized similarly, except that the fluid state is not simply mirrored at the interface but replaced, for example, with a predefined state describing the inflow conditions into the simulated volume. For hydrodynamics, periodic, reflective, or inflow/outflow boundaries can be independently selected for each spatial dimension.

6. Additional Physics

For a number of astrophysical systems, modeling just gravity and MHD already provides an interesting approximation that warrants detailed study. However, there are a number of problems where this alone is only a starting point and further important physics needs to be added. This can, for example, be introduced through source or sink terms of thermal energy due to interaction with radiation fields or local energy production in nuclear reactions. As the precise nature, and thus the implementation, of such terms depends highly on the system under study, and the verification of the corresponding calculations is far less clear than in the gravity+MHD case, we include only two very simple modules for such extra physics here to provide an illustrative guideline for how more sophisticated modules can be implemented.

The first example is a sink term due to radiative cooling, and the second is a simple sub-grid model for the interstellar medium and its embedded star formation. Both of these models have been extensively applied in the literature to simulations of galaxy formation, but recent work has largely moved on to more sophisticated treatments. All equations in the following section are in proper, not co-moving coordinates, and are also implemented in this way in the code.

6.1. Radiative Cooling

One of the key additional physics ingredients to an optically thin astrophysical plasma is radiative cooling. In AREPO, this is implemented as a sink term in the energy equation as described in Equation (18). The main challenge with radiative cooling is that this loss term is a strong function of metallicity and

density, making the cooling rate a stiff equation that can be difficult to integrate numerically. In order to avoid tight time step constraints that an explicit integration scheme for radiative cooling would impose, AREPO uses a first-order implicit integration of the cooling term, based on a root-finding algorithm of the equation

$$u' - u_n - \frac{\Lambda(u')}{\rho} \Delta t = 0. \quad (42)$$

The solution for u' at the end of a step is found iteratively, evaluating the cooling function Λ at the respective specific internal energy and ionization state for the next iteration step. In the public version of the code, only a primordial cooling network for H and He (Cen 1992; Katz et al. 1996) under the assumption of collisionless ionization equilibrium is included; however, extensions to this model are straightforwardly possible.

6.2. Modeling Star Formation

Upon including gas cooling in hydrodynamical simulations of galaxy formation, one quickly arrives at situations where the gas loses its pressure support and collapses under the relentless pull of gravity. In reality, this gravitational collapse is very complex and occurs over roughly 10 orders of magnitude before local fragments give birth to individual stars. In simulations, however, it is both impractical and currently impossible to follow the gravitational collapse of a molecular cloud to all of the stars it forms and the onset of nuclear reactions in them, let alone in an entire galaxy. Instead, one needs to resort to much more simplified treatments, taking the form of so-called sub-grid models. They stand for the general idea of introducing an effective model for complex unresolved astrophysical processes, which is often necessary when the physics cannot be resolved directly, but it comes at the price of adding uncertainty and heuristic input.

For example, one can define, as is commonly done, a density threshold above which gas forms so-called star particles with an estimated mean rate. The star particles represent stellar populations, and they are collectively modeled as a population of collisionless particles that, in particular, no longer interact with the surrounding gas via hydrodynamical forces. The implemented example of star formation in the public release of AREPO follows the Springel & Hernquist (2003) model for treating the unresolved interstellar medium. We note, however, that the metal enrichment and wind formation models (Sections 5.3 and 5.4 of Springel & Hernquist 2003) are not part of this implementation.

The basic idea of this model is to not attempt to resolve the multiphase gas structure in the ISM, but instead represent its spatially averaged behavior through a simple smooth model. The ISM is pictured as a two-phase medium in pressure equilibrium, consisting of cold clouds in which star formation can occur and which are embedded in a hot, space-filling medium. For a detailed discussion of the model, we refer to Springel & Hernquist (2003), and only list the primary implemented equations and their connection to the free parameters of the model here. In particular, the effective specific thermal energy of a star-forming cell, i.e., a cell with

Table 3
Code Parameters for Interstellar Medium and Star Formation Modeling

Description	Symbol	Code Parameter
Gas overdensity in cosmological runs above which star formation is possible	$\rho_{\text{c.o.d.}}$	CritOverDensity
Threshold temperature below which star formation is possible	T_{thresh}	TemperatureThresh
Threshold density above which model is applied	ρ_{th}	CritPhysDensity
Fraction of stars that go off as supernovae	β	FactorSN
Supernova evaporation parameter	A_0	FactorEVP
Temperature of supernova in two-phase ISM model	T_{SN}	TempSupernova
Temperature of cold phase of two-phase ISM model	T_{c}	TempClouds
Reference star formation time (= timescale at critical density)	t_0^*	MaxSfrTimescale

density exceeding ρ_{th} , is given by

$$u_{\text{EoS}} = u_{\text{hot}}(1 - x) + u_{\text{c}}x, \quad (43)$$

with x being the mass fraction of cold clouds, calculated as

$$x = 1 + \frac{1}{2y} - \sqrt{\frac{1}{y} + \frac{1}{4y^2}}, \quad (44)$$

$$y = \frac{t_*}{t_{\text{cool}}} \frac{u_{\text{hot}}}{\beta u_{\text{SN}} - (1 - \beta)u_{\text{c}}}. \quad (45)$$

Here β is the fraction of stars according to the initial mass function that explode as supernovae, and t_{cool} is the cooling time derived from the employed cooling model. The specific internal energies can be associated with the temperature parameters via

$$u_{\text{c}} = \frac{k_B}{(\gamma - 1) \mu_{\text{neutral}} m_p} T_{\text{c}}, \quad (46)$$

$$u_{\text{SN}} = \frac{k_B}{(\gamma - 1) \mu_{\text{ionized}} m_p} T_{\text{SN}}, \quad (47)$$

where a completely neutral and fully ionized gas is assumed, respectively. The specific energy in the hot ISM phase is given by

$$u_{\text{hot}} = \frac{u_{\text{SN}}}{1 + A(\rho)} + u_{\text{c}}, \quad (48)$$

and the supernova evaporation parameter is given by

$$A(\rho) = A_0 \left(\frac{\rho}{\rho_{\text{th}}} \right)^{-\frac{4}{5}}. \quad (49)$$

Finally, the star formation timescale is

$$t_*(\rho) = t_0^* \left(\frac{\rho}{\rho_{\text{th}}} \right)^{-\frac{1}{2}}, \quad (50)$$

where star formation is only allowed to take place if the temperature is below T_{thresh} and the density is above ρ_{th} and $\rho_{\text{c.o.d.}}$. In this case, the star formation rate of a cell is taken to be

$$\dot{m}_* = \frac{m_{\text{cell}}}{t_*(\rho)}, \quad (51)$$

and actual star particles are spawned stochastically with this rate. If a star formation event occurs and a cell has too little mass left afterwards, the cell is converted to a star particle in full; otherwise, it survives with correspondingly reduced mass, energy, and momentum content. The free parameters of this model are summarized in Table 3.

7. Time Integration

For gravity and ideal MHD, AREPO uses explicit time integration, which puts certain constraints on the size of individual local time steps both with respect to the accuracy and stability of the scheme. Every dark matter particle has its own individual local gravitational time step constraint. For each cell, there are separate time step constraints for hydrodynamics, gravitation, and potentially for source and sink terms. Generally, the most restrictive constraint is applied for all of the calculations of a single cell.

7.1. Time Step Constraint

For gas cells, we define a local Courant–Friedrichs–Levy (CFL) time step criterion

$$\Delta t \leq C_{\text{CFL}} \frac{r_{\text{cell}}}{v_{\text{signal}}}, \quad (52)$$

with the Courant factor C_{CFL} as a free parameter and the cell radius given by

$$r_{\text{cell}} = \left(\frac{3V}{4\pi} \right)^{1/3}, \quad (53)$$

where V is the volume of the Voronoi cell. The signal speed, in the case of a moving mesh, is calculated taking into account sound speed and Alfvén speed

$$v_{\text{signal}} = \left(\gamma \frac{p}{\rho} + \frac{B^2}{\rho} \right)^{1/2}. \quad (54)$$

In the case of a static mesh, the gas bulk velocity is added to the signal speed, which leads to a more restrictive time step criterion compared to the moving mesh. For MHD, the Powell cleaning scheme imposes an additional constraint,⁷

$$\Delta t_{\text{cleaning}} = \frac{B + \sqrt{0.02 u_{\text{th}} \rho}}{2|\nabla \cdot \mathbf{B}| |\mathbf{v}|}. \quad (55)$$

If the star formation model is active, we adopt a further constraint set by the ratio of mass of a gas cell m and its star formation rate $\dot{m}_{\text{SF}}$,

$$\Delta t_{\text{SF}} = 0.1 \frac{m}{\dot{m}_{\text{SF}}}. \quad (56)$$

⁷ Note that formally the Powell cleaning scheme does not require additional time step constraints. The stated constraint is adopted to ensure the code functionality is unaltered from the version used in many production runs.

Table 4
Code Parameters for Time Step Constraints

Description	Symbol	Code Parameter
Gravity time step parameter	C_{grav}	<code>ErrTolIntAccuracy</code>
Courant factor for MHD	C_{CFL}	<code>CourantFac</code>

For gravitational accelerations, we adopt the criterion

$$\Delta t \geq \sqrt{\frac{2C_{\text{grav}}\epsilon_{\text{soft}}}{|\mathbf{a}|}}, \quad (57)$$

with C_{grav} being a free parameter. Additionally, a nonlocal magnetohydrodynamic time step criterion (Springel 2010b; Gnedin et al. 2018) is adopted (`TREE_BASED_TIMESTEPS`)

$$\Delta t_i \geq \min_{j \neq i} \left(\frac{r_{ij}}{c_i + c_j - \mathbf{v}_{ij} \cdot \mathbf{r}_{ij}/r_{ij}} \right), \quad (58)$$

which is meant to preempt the earliest arrival time of hydrodynamical waves originating in any other cell. It is implemented in practice using a tree structure to evaluate it efficiently, avoiding the need to loop over all particles in the time step calculation. The parameters for the time step constraints are listed in Table 4.

7.2. Local Time Stepping

To allow for each element to be integrated with an as large as possible but as small as necessary time step, while still maintaining a high degree of synchronization, AREPO uses the common approach of a power-of-two time step hierarchy. The total simulation time is subdivided into 2^N equal steps. The corresponding time step size for a time bin is thus $\Delta t = (t_{\text{end}} - t_{\text{start}})/2^N$. Particles are then associated with the time bin that corresponds to the largest time step that is just smaller than the most restrictive time step constraint from gravity and MHD. A linked list of particles belonging to each time bin is kept to be able to access particles in a given time bin efficiently without having to search for them with a loop over all particles. When a computational element has finished a time step, it may change its time bin. Transitioning to a shorter time step is always possible, but changing to a longer time step only occurs if the current time is synchronized with the target time bin, i.e., it is ensured that shorter time steps always stay nested within longer ones throughout the whole hierarchy.

7.3. Time Stepping Gravity

The gravitational time integration is done with a second-order accurate leapfrog scheme, expressed through alternating “drift” (which modify the positions) and “kick” (which modify the velocities) operations. For fixed time step sizes, this results in a symplectic integration scheme. The particular implementation is very similar to that of the GADGET-2 code (Springel 2005).

Normally, an active particle receiving a kick interacts with the full mass distribution, independent of its time bin. This breaks manifest momentum conservation when local time steps are used and necessitates a full tree construction even if only a small fraction of particles requires force calculations, negatively impacting performance for deep time step hierarchies. This can be addressed with an alternative hierarchical time

integration approach (`HIERARCHICAL_GRAVITY`), in which the gravitational Hamiltonian is systematically split, such that shorter time bins are evolved with their own part of the Hamiltonian only. The particular implementation of this idea as adopted in AREPO is based on Pelupessy et al. (2012), with full details given in V. Springel et al. (2020, in preparation).

7.4. Time Stepping the Finite-volume Scheme

The time integration of the hydrodynamic quantities is described in Pakmor et al. (2016b), and differs slightly from the original implementation of a MUSCL-Hancock scheme in Springel (2010b). In particular, now a scheme more similar to Heun’s method for time integration is used, ensuring that only a single mesh construction is required for each time step while obtaining formal second-order convergence in time. One key difference to static grid codes is that in a moving mesh, the facet areas $A_{ij} = A_{ij}^n$ as well as the cell volumes $V_i = V_i^n$ are functions of time, and higher-order schemes need to take this time dependence into account. The update of the volume integrated conserved variables of cell i , $Q_i = \int_{\text{cell } i} U dV$, with U being the vector of conserved quantities, is computed as

$$Q_i^{n+1} = Q_i^n - 0.5\Delta t \times \left(\sum_j A_{ij}^n F(W_{ij}^n, W_{ji}^n) + A_{ij}^{n+1} F(W_{ij}', W_{ji}') \right), \quad \text{with} \quad (59)$$

$$W_{ij}^n = W_i^n + \mathbf{d}_{ij} \cdot \frac{\partial W_i}{\partial \mathbf{x}}, \quad (60)$$

$$W_{ij}' = W_i^n + \mathbf{d}_{ij}' \cdot \frac{\partial W_i}{\partial \mathbf{x}} + \Delta t \frac{\partial W_i}{\partial t}, \quad (61)$$

where $\mathbf{d}_{ij}$ is the position vector of the geometric center of the interface between cells i and j relative to the center of cell i . The update is thus a combination of fluxes computed at the beginning of the current time step and fluxes computed on an updated mesh at the end of the time step using time-extrapolated quantities indicated by superscript $'$. The flux calculation itself and the associated Riemann problems are hence evaluated twice in each time step. The time derivatives are estimated via the spatial gradients using the continuity equation (Pakmor et al. 2016b, their Equations (7)–(9)). Local time stepping is realized in a conservative manner by adding fluxes to Q in a pairwise manner to the two cells on either side of an active interfaces, i.e., whenever at least one of the involved cells is active (Springel 2010b, Section 7). The time integration of the hydrodynamics is illustrated in Figure 3.

8. Domain Decomposition and Load Balancing

A key feature of modern parallel simulation codes is their ability to distribute computations over a large number of distributed-memory compute nodes, thereby allowing very large computations on modern supercomputers. A prerequisite to make this possible is however that both the computational as well as the memory load must be balanced well across the individual nodes and processes. To achieve this, AREPO uses the MPI standard for distributed-memory parallelization, and executes a domain decomposition routine to subdivide the simulated volume into a set of disjoint pieces that are mapped onto the MPI ranks, allowing data replication to be kept to a minimum.

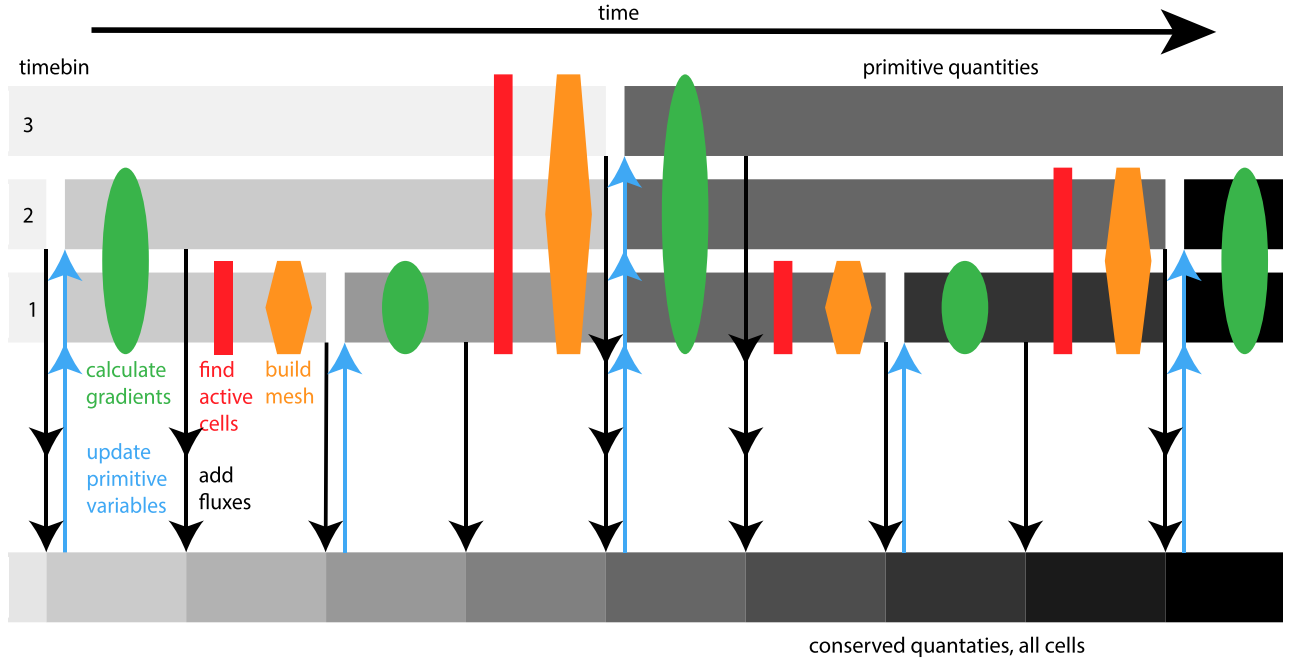


Figure 3. Illustration of the time evolution of the Euler equations using local time stepping (in this case, only three time bins are sketched, for simplicity) and the roles of primitive and conserved quantities. For cells of each time bin, time evolution follows the same steps: update of the primitive variables, calculation of gradients, first flux calculation, determination of mesh at the end of the time step, and a second flux calculation. Fluxes through interfaces of cells on different time steps are added to both cells’ conserved quantities during the smaller time step. This leads to partially updated states in conserved quantities of cells in larger time bins.

The domain decomposition is redone for every time step whenever at least a specified fraction of particles $f_{\text{domain}} = N_{\text{active}}/N_{\text{total}}$ is active. In order to allow for better simultaneous balancing of different cost factors (which are the CPU-time needed to compute gravity and hydrodynamics, possibly for several different time bins, and the memory load for particles and cells), the computational domain is split into N_{domain} chunks,

$$N_{\text{domain}} = n_{\text{domain}} N_{\text{MPI}}, \quad (62)$$

where $n_{\text{domain}} \geq 1$ is a free integer parameter, and N_{MPI} is the number of MPI ranks. By “oversampling” the number of MPI ranks with n_{domain} larger than unity, one gives the code the ability to map several chunks onto the same MPI rank in order to more effectively smooth out residual imbalances. This allows us, for example, to combine on one MPI rank a chunk that has a lot of work but few particles with one that has many particles but requiring little work, simultaneously balancing the memory and the work load.

In practice, AREPO carries out the domain decomposition by first constructing the highest levels of the oct-tree covering the full simulation volume. The leaf nodes of this “top-level tree” are iteratively replaced by refined leaf nodes if their load exceeds a target fraction

$$w_{\text{opening}} = \frac{w_{\text{total}}}{N_{\text{domain}} m_{\text{domain}}} \quad (63)$$

of the total load w_{total} . Here the factor m_{domain} is usually set to a value of a few (~ 2.5 – 4.0) to yield a sufficiently fine discretization of the load at the level of the leaf nodes, so that adjacent ones⁸ can be combined into a final set of N_{domain}

chunks that have, to a good accuracy, very similar loads. These domain chunks are finally mapped to the N_{MPI} processes, where for $n_{\text{domain}} > 1$ additional balancing opportunities are realized. This secondary balancing step was not realized in the GADGET-2 code.

To estimate the work-related CPU-cost associated with particles and cells in a leaf node, AREPO uses cost factors associated with them. Specifically, to estimate the computational cost of the hydrodynamics calculations associated with a domain patch,

$$c_{\text{hydro}} = \sum_i b_i \quad (64)$$

is used, where i runs over all particles located in the domain piece, and the factor b_i counts how often the particle will be active in traversing the time step hierarchy before the next domain decomposition is expected to take place. For the gravity calculations, a similar formula is used,

$$c_{\text{grav}} = \sum_i g_i b_i, \quad (65)$$

but here also a cost factor g_i enters that measures the number of particle–cell interactions that need to be evaluated for the particle in the most recent gravity calculation. Depending on clustering state, the values for g_i can vary significantly, whereas a hydrodynamical cell always has a very similar number of neighbors on average. Finally, for the memory load, each collisionless particle and hydrodynamical cell contribute equally in their respective category. The parameters of the load-balancing algorithm are listed in Table 5.

9. Other Code Features

Besides the core functionality to advance a simulation in time, AREPO has a number of additional features that influence

⁸ Here adjacency is understood in a Peano–Hilbert sense.

Table 5
Code Parameters for Load Balancing

Description	Symbol	Code Parameter
Domain decomposition frequency	f_{domain}	ActivePartFracForNewDomainDecomp
Number of domains per MPI task	n_{domain}	MultipleDomains
Depth parameter of the top-level tree	m_{domain}	TopNodeFactor

practical aspects of how the code can be used in specific situations. These are described in the following.

9.1. On-the-fly Structure and Substructure Finding

AREPO includes a friends-of-friends (FOF) and a substructure identification algorithm that can be used both on-the-fly or in post-processing. The algorithms are originally described in Springel et al. (2001), and can be optionally called before each snapshot dump, or in other intervals.

In brief, group and subhalo identification works in the following way. First, an FOF algorithm is applied to the particle distribution to define groups as equivalence classes, where any pair of particles is in the same group if they are closer to each other than l_{FOF} times the mean inter-particle separation. Next, these groups are subjected to substructure identification with the SUBFIND algorithm (Springel et al. 2001).

To this end, the local density is estimated in an SPH-like approach at the position of all member particles, by using a smoothing length that encloses $N_{\text{ngb,sub}}$ (weighted) nearest neighbors. Then, an excursion set algorithm is used to identify locally overdense regions as substructure candidates. Each candidate is then treated with a gravitational unbinding procedure that iteratively excludes all particles with positive total energy from the member list of the substructure candidate. The potential binding energy is calculated using a tree algorithm analogous to the one used for the gravitational force calculation, with opening criterion ϑ_{sub} . If a gravitationally bound set of particles survives that contains at least a minimum number of particles, the substructure is retained in the list of final subhalos. The algorithm can detect nested sets of subhalos, but each particle/cell is counted only toward the mass of one substructure. Groups and subhalos are allowed to be much larger than the ones that fit into the memory of a single MPI rank.

The particles and cells of a snapshot dump are automatically stored on-disk such that particles belonging to individual (FOF) groups are grouped in the output as one block, in order of descending group size. Within a given group, the particles are further sorted according to subhalo membership, again in descending order of subhalo size. Finally, within a subhalo, the particles are sorted by their binding energy, with the particle with the lowest total energy coming first. This allows efficient random access to the particle data belonging to groups or subhalos even of very large simulation outputs. More details on the structure of the simulation output when using SUBFIND can be found in the data release papers of the Illustris and IllustrisTNG projects (Nelson et al. 2015, 2019b). The parameters for the structure and substructure finding algorithms are listed in Table 6.

9.2. Initial Conditions Conversion

Due to its ancestry in the SPH code GADGET, AREPO includes an option to convert SPH initial conditions to ones

Table 6
Code Parameters for Structure Finding

Description	Symbol	Code Parameter
FOF linking length	l_{FOF}	FOF_LINKLENGTH
Number of neighbors for subfind density estimate	$N_{\text{ngb,sub}}$	DesLinkNgb
Tree opening criterion in subfind	ϑ_{sub}	ErrTolThetaSubfind

suitable for grid-based calculations. To this end, first an oct-tree structure is built for the computational domain using the SPH particles, augmented with a coarse grid of massless background particles to fill the computational volume of an enclosing box. The leaf nodes of this tree form a tessellation of space, which is now used for distributing the SPH quantities in a conservative fashion to the leaf nodes using the SPH kernel. Finally, a mesh-generating point is placed at the cell-centers of the leaf nodes, inheriting the conserved fluid quantities that were scattered by the SPH particles to these points. In this way, grid-based initial conditions for AREPO are generated that mimic the SPH distribution as closely as possible, but without having a highly irregular initial mesh, which would result if one uses the SPH particles directly (which is also possible).

One of the provided examples included with AREPO, a merger of two galaxies, demonstrates how this is done in practice. We want to emphasize, however, that the initial conditions created in this way may still suffer from Poisson-like discreteness noise of SPH snapshots, and directly creating new, grid-based initial conditions for a known smooth hydrodynamic field will result in higher-quality initial conditions. Therefore, whenever possible, we recommend to rather modify the initial conditions generating code to support grid-based setups over converting SPH initial conditions.

9.3. Input and Output

The standard input and output of simulation time slices is done via snapshot files, usually employing the HDF5 format.⁹ Older binary formats identical to the ones used in GADGET-2 (referred to as format 1 and 2) are also supported. Both initial conditions and snapshot outputs can be distributed over an arbitrary number of files.¹⁰ The number of desired files is recognized automatically for inputs, while for outputs it is specified as a parameter, which can be chosen both for convenience (to limit the maximum size of individual files, which helps in transferring/archiving large snapshots) and for increasing the achieved I/O bandwidth, depending on the system. This is because the files belonging to an input/output set can be read/written in parallel, with each file being assigned to a specific MPI task, normally chosen automatically to be maximally spread over the used compute nodes as well. These

⁹ <https://www.hdfgroup.org/solutions/hdf5/>

¹⁰ For outputs, the number is limited to be at most equal to the number of employed MPI ranks.

tasks then handle the I/O for a group of MPI ranks and collect/send the corresponding data over the MPI communication fabric. In this way, highly efficient parallel I/O can be achieved that makes full use of the maximum I/O bandwidth available on a particular parallel file system. In order to not overload a given system (and to retain responsiveness for other users), one can also specify a maximum number of files written/read in parallel at any given time.

Note that in this approach, each individual file is written with ordinary POSIX semantics, i.e., AREPO does neither make use of the parallel I/O functionality of HDF5 nor employs MPI-IO functions. The explicit parallel I/O approach we prefer minimizes side effects of these complex libraries, such as their internal memory allocation calls, which can become prohibitive when physical memory is extremely scarce, a situation that (unfortunately) often occurs in practice for large production runs.

9.3.1. Binary Dumps for Restarting the Code

Since the run time of simulations is often longer than the wall-clock time limits of queuing systems on compute clusters, a single submission of AREPO may not be enough to finish a simulation. The code therefore includes a check-pointing functionality, where so-called restart files are written by each MPI rank separately, allowing a seamless continuation of a run in a subsequent submission. Particular care has been taken to ensure that resuming from such a restart dump yields binary-identical results compared to continuously executing the code without a break (changing the number of MPI ranks upon restarting is thus not possible).

The restart files are essentially binary dumps of all necessary simulation variables in their full precision. They are written automatically at the end of a specified run time, and also in regular time intervals to guard against losing time due to a node or code failure. Copies of the two most recent restart files are kept automatically, and replaced in a cyclic fashion.

For large, long-running simulations (such as IllustrisTNG), the cumulative data volume written to disk can become extremely large, to the point that potential file system errors can be no longer ignored, despite them being exceedingly rare. To this end, AREPO makes special efforts to detect and avoid these for the critical restart files. After writing a set of restart files, md5-checksums of the data in memory are compared with md5-checksum of the file on-disk when read-in by an MPI rank on a *different* compute node. We found this to be a reliable way to detect corrupted files, in which case the code will terminate to ensure that the user is made aware of the file system problems, and in order to leave one of the two sets of restart files on-disk intact.

9.4. Memory Management

Computational power tends to increase faster than the amount of physical memory available on modern computers. Cosmological simulations are therefore often memory bound and not necessarily CPU-time bound. It is therefore important to make optimal use of the available physical memory. This, in particular, calls for complete control over the amount of memory used by the simulation code at any given time, the safe prevention of any memory leak, and the avoidance of fragmentation of the memory heap.¹¹ In AREPO, this is

achieved by an internal memory manager that, upon start-up, allocates a single block of memory from the system for each MPI task, and then handles all of the dynamic memory allocation requests internally using this storage space. In this way, the maximum amount of physical memory available to application codes can robustly be used by AREPO (modulo the sometimes uncertain requirements of the MPI library, or of background system processes related to, e.g., the Lustre parallel file system). In case one of the MPI ranks running out of memory, AREPO itself terminates and provides a verbose overview of the allocated memory, including information about which line of the code allocated each array or variable.

In order to avoid fragmentation, the internal memory manager normally enforces that allocated memory blocks are freed in inverse order of the allocation, such that it is used like a stack. However, movable memory blocks can also be used in special situations where this leads to simpler or more efficient code, so that the memory manager does not impose a serious restriction on the programming model.

9.5. Usage of Libraries

The usage of libraries in codes can reduce the development time and improve the reliability of individual code parts. An overly excessive use of (exotic) libraries, however, can also cause problems for building and maintaining the code, and it introduces a source of potential errors and performance problems outside of the direct control of the simulation code developers. In AREPO, the general philosophy is, for this reason, to limit the number of libraries used to an essential minimum. In practice, this means that AREPO makes use only of common and well-tested libraries that are usually already present on compute clusters and supercomputers. These are MPI for parallelism, HDF5 for I/O, GSL for a few elementary numerical integrations and pseudo random number generation, FFTW for FFTs, and finally the GMP library for arbitrary precision integer arithmetic. The latter is employed for robustly resolving degeneracies in the mesh construction.

AREPO is written in C, using the C11 standard. Consequently, it can be compiled with a large number of different compilers. We recommend the free and widely used GNU compiler for its excellent robustness and speed. For AREPO, our experience is that commercial compilers only sometimes result in slightly better performance of the code on some machines, but are also more prone to producing incorrect code optimizations when aggressive optimization levels are used. The code employs the MPI-2 standard for distributed-memory parallelism, and exercises only a comparatively small subset of all possible MPI functions. In the public version of AREPO, only communication patterns that, in our experience, are very reliable on most machines are used.

10. Examples

AREPO includes a number of small example simulations that are also used for code verification. They are implemented in such a way that they can be (and are) used for regular automated code regression tests to ensure that the functionality and correctness of the code under certain conditions, and especially after changes have been made, is not broken. The default set of tests is listed in Table 7.

¹¹ Fragmentation can especially become an issue on compute nodes with a simplified virtual memory allocation system, such as the Bluegene.

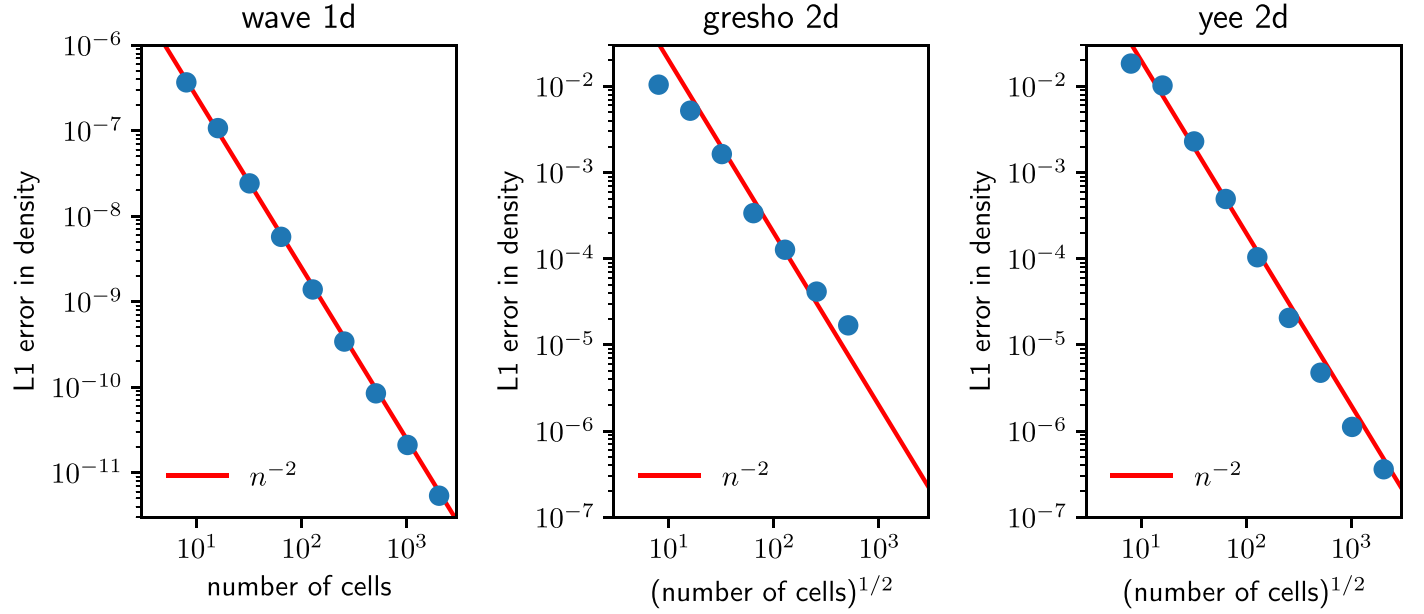


Figure 4. Convergence of the density field for the wave problem (left), the Gresho vortex (center), and the Yee vortex (right). Wave propagation and the Yee vortex show second-order convergence, the Gresho vortex shows lower-than-second-order convergence, in qualitative agreement with previous findings (Springel 2010b). This is likely caused by the non-smooth initial conditions of this problem.

Table 7
Implemented Simulation Examples

Name of Example	Description	Verification
wave_1d	One-dimensional wave propagation	Analytical solution
shocktube_1d	One-dimensional Riemann problem	Exact solution
interacting_blastwaves_1d	One-dimensional interacting shocks	High-resolution simulation
mhd_shocktube_1d	One-dimensional MHD Riemann problem	Exact solution
polytrope_1d_spherical	Hydrostatic $n = 1$ polytrope	Initial conditions
noh_2d	Converging flow in 2D	Analytic solution
gresho_2d	Stationary vortex	Initial conditions
yee_2d	Stationary vortex	Initial conditions
current_sheet_2d	Magnetic field reversal	Magnetic energy
noh_3d	Converging flow in 3D	Analytic solution
cosmo_box_gravity_only_3d	Gravity-only cosmological structure formation	Halo mass (reference run)
cosmo_box_star_formation_3d	Star formation in cosmological volume	Stellar mass density (reference run)
cosmo_zoom_gravity_only_3d	Gravity-only cosmological zoom simulation	Subhalo mass (reference run)
galaxy_merger_star_formation_3d	Merger of two isolated disk galaxies	Stellar mass (reference run)
isolated_galaxy_collisionless_3d	Isolated disk galaxy (stars and dark matter)	Disk scale height

10.1. Wave Propagation

One of the most elementary tests for a hydrodynamic code is the propagation of a small amplitude sound wave in one dimension. We use a 1D periodic domain of length $L = 1$, filled with a fluid of density $\rho_0 = 1$, velocity $v_0 = 0$, pressure $p_0 = 3/5$, and an adiabatic index of $\gamma = 5/3$. We then introduce a small sinusoidal adiabatic perturbation of amplitude $\delta = 10^{-6}$ and wavenumber $k = 2\pi/L$, such that density, velocity, and specific internal energy are given by

$$\rho(x) = \rho_0[1 + \delta \sin(kx)], \quad (66)$$

$$v(x) = 0, \quad (67)$$

$$u(x) = u_0[1 + \delta \sin(kx)]^{\gamma-1}, \quad (68)$$

where x is the spatial coordinate. We use $C_{\text{CFL}} = 0.3$ and simulate the problem until $t = 1$, i.e., the time the perturbation takes to propagate once through the computational domain, and

compare the result with the initial conditions. Since this is a smooth problem, it can also be used to test the convergence order of the hydrodynamics scheme, which, in our case, is particularly sensitive to the gradient estimates and shown in Figure 4, left panel.

10.2. Riemann Problem

Another basic test is the calculation of a one-dimensional Riemann problem, i.e., a domain split into a left-hand initial state and a right-hand state. In our particular example, we set up a density and pressure jump with no initial velocity on either side of the initial separation at $x = 10$, using a box with domain length $L = 20$ and reflective boundary conditions. We use a moving mesh with 128 initially equally spaced cells, without refinement or de-refinement for this test. The density ρ , velocity v , and pressure p to the left and right of the discontinuity are

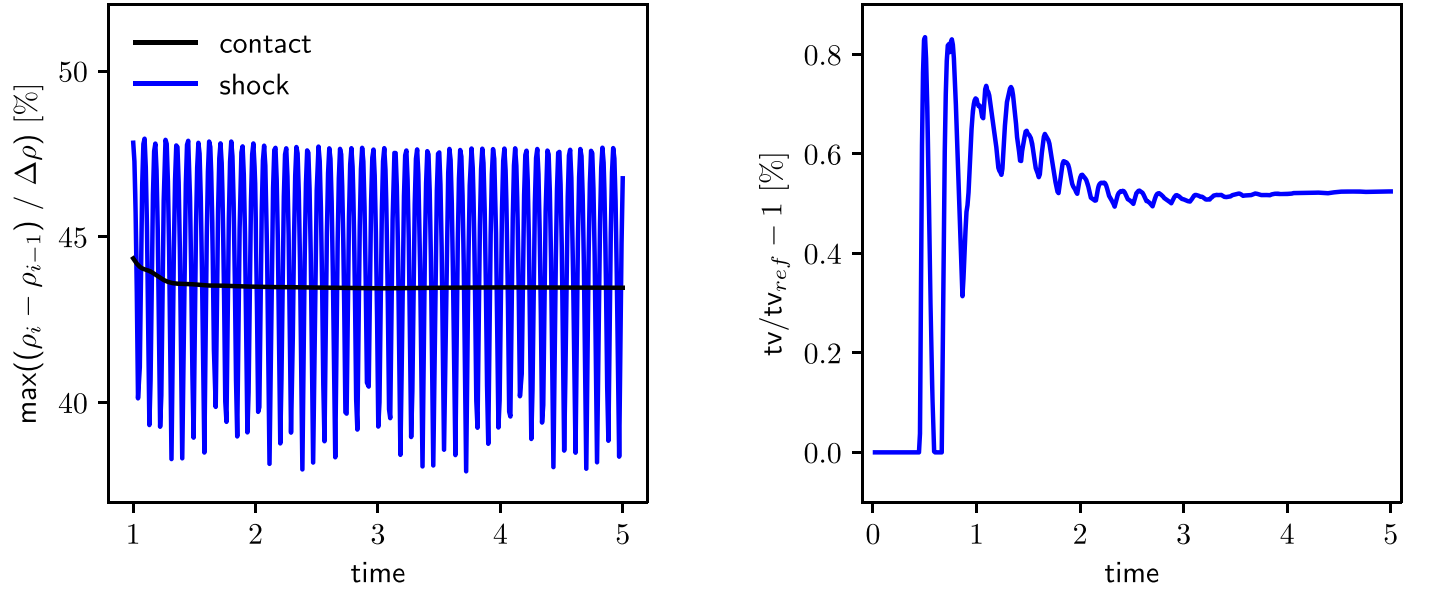


Figure 5. Fraction of the discontinuity resolved by a single cell (left) for the 1D shock tube test. The oscillation in the shock is a result of cells moving through the shock interface, while the constant value at the contact discontinuity is due to cells moving with the flow. The flux limiters cause the total variation to increase by less than 1 % (right), effectively avoiding spurious oscillations at discontinuities.

given by

$$\rho_L = 1.0, \quad \rho_R = 0.125, \quad (69)$$

$$v_L = 0, \quad v_R = 0, \quad (70)$$

$$p_L = 1.0, \quad p_R = 0.1, \quad (71)$$

respectively, an adiabatic index of $\gamma = 1.4$ is assumed (see Sod 1978), and $C_{\text{CFL}} = 0.3$. These initial conditions lead to a shock traveling to the right, a rarefaction wave traveling to the left, and a moving contact discontinuity in the center. In this test, we check not only against the iteratively computed solution, but also monitor the total variation of the solution, i.e., the sum of absolute differences in the hydrodynamical quantities of neighboring cells, to ensure that the slope limiters prevent oscillatory instabilities from growing. Similarly, we analyze the physical discontinuities and check that the jumps in density, velocity, and pressure, are resolved with at most a few cells across (four has proven to be a suitable value for demanding that the 5th and 95th percentiles of the jump are caught). This sets strong limits on the allowed numerical diffusivity of the hydrodynamical scheme. Figure 5 shows the maximum change in density from a single cell to the next relative to the analytic discontinuity (left) as well as the total variation of the density relative to the analytic solution (right) as a function of time.

10.3. Interacting Blastwaves

A slightly more complex, and also numerically more challenging, test is a problem of interacting shocks as described, e.g., in Springel (2010b). We set up a 1D domain with length $L = 1$ and reflective boundary conditions. Initially, two Riemann problems are set up at $x = 0.1$ and $x = 0.9$, leaving three hydrodynamic states in the initial conditions. We choose as density, velocity, and pressure, for the left, center, and right states:

$$\rho_L = \rho_C = \rho_R = 1, \quad (72)$$

$$v_L = v_C = v_R = 0, \quad (73)$$

$$p_L = 1000, \quad p_C = 0.1, \quad p_R = 100, \quad (74)$$

respectively. The adiabatic index of the gas is $\gamma = 1.4$. We allow the initially equally spaced cells to move with the flow, but do not include refinement or de-refinement. As in Springel (2010b), the problem is evolved until $t = 0.038$ using $C_{\text{CFL}} = 0.3$, and at this time is compared to a very high-resolution, static grid solution of the same initial conditions (see Figure 6).

10.4. MHD Shocktube

To test the MHD Riemann solver in AREPO, we use a magnetic Riemann problem as described in Torrilhon & Balsara (2004) and Fromang et al. (2006). We employ a 1D domain with length $L = 2.5$ and reflective boundary conditions. We set up a discontinuity at $x = 1$ with density, velocity, pressure, and the three components of the magnetic field to the left and right, given as

$$\rho_L = 1.0, \quad \rho_R = 0.2, \quad (75)$$

$$v_L = 0.0, \quad v_R = 0.0, \quad (76)$$

$$p_L = 1.0, \quad p_R = 0.2, \quad (77)$$

$$bx_L = 1, \quad bx_R = 1, \quad (78)$$

$$by_L = 1, \quad by_R = \cos(\alpha), \quad (79)$$

$$bz_L = 0, \quad bz_R = \sin(\alpha), \quad (80)$$

respectively.¹² In this example, we use $\alpha = 3$, and the adiabatic index of the thermal fluid is $\gamma = 5/3$. We allow the initially uniform mesh to move with the fluid flow, but refrain from using refinement or de-refinement. The employed Courant factor is $C_{\text{CFL}} = 0.3$. The solution is verified at time $t = 0.4$ against the exact solution (Figure 7). We additionally perform the same simulation using the ATHENA++ code (White et al. 2016;

¹² Note that the magnetic field is given here in Heaviside–Lorentz units, while in the initial conditions, Gaussian units are used.

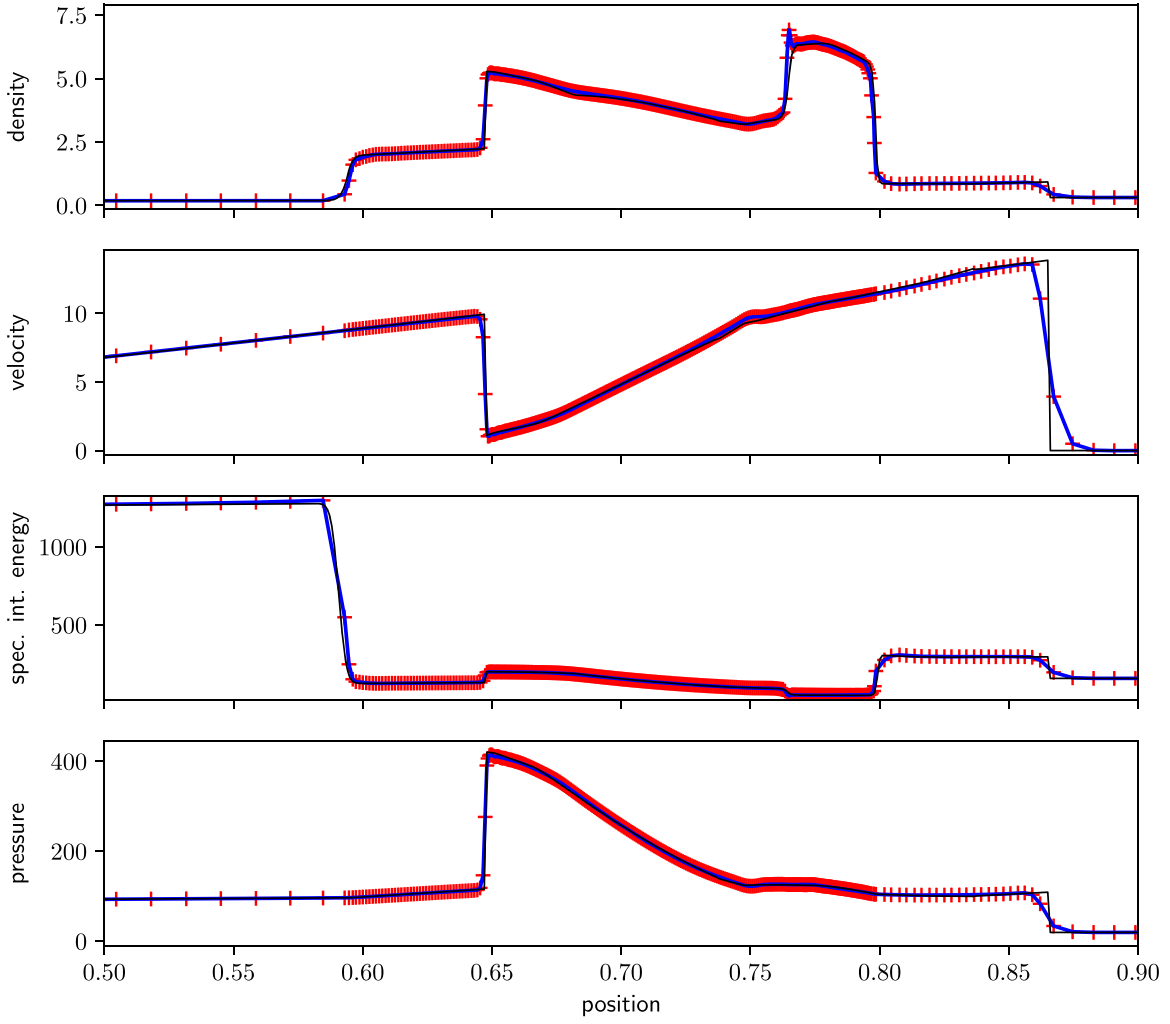


Figure 6. Hydrodynamic quantities of an interacting blast wave problem at $t = 0.038$. The black line shows the high-resolution reference solution, and the blue line shows the moving-mesh solution with the red crosses denoting the cell positions.

Felker & Stone 2018; J. Stone et al. 2020, in preparation).¹³ Similarly to Fromang et al. (2006), the low-resolution version shows significant errors, and only when the resolution is increased substantially can the exact solution be recovered (Figure 8). We find the same behavior both for AREPO and ATHENA++; however, roughly a factor of three higher resolution is required in AREPO compared to ATHENA++ to produce a similar degree of accuracy in this problem.

10.5. Static Polytrope

To test the 1D spherically symmetric mode of AREPO, we set up a known hydrostatic solution of the $n = 1$ polytrope. This setup uses a reflective inner and an open outer boundary condition with a computational domain from $r = 0.1$ to $r = 1$. For the $n = 1$ polytrope (with $\gamma = 2$), a static solution

$$\theta(\xi) = \frac{\sin(\xi)}{\xi} \quad (81)$$

exists, with $\xi = r/r_{\text{scale}}$ being the dimensionless radius and $\rho = \rho_0 \theta$. In this particular setup, we use $r_{\text{scale}} = 0.8/\pi$,

$\rho_0 = 1$, and $G = 1$, in simulation units. The pressure is given at each radius by $p = K \rho^\gamma$ with $K = 2\pi G r_{\text{scale}}^2$, and the initial velocity is $v = 0$ everywhere. For gravity, besides the gravitational force of the enclosed gas mass, we include a static core with radius $r = 0.1$ and mass $m = 4.18 \times 10^{-3}$. The employed Courant factor is $C_{\text{CFL}} = 0.3$, and the mesh is kept static in this problem. Since this problem is set up in hydrostatic equilibrium, verification is done by comparing the result at $t = 1$ to the initial conditions (see Figure 9).

10.6. Magnetic Current-sheet Problem

Another test of the MHD scheme, in particular for the numerical reconnection rates, is the so-called current-sheet problem, following Gardiner & Stone (2005). We set up a two-dimensional domain with side length $L = 2$ and periodic boundary conditions. The magnetohydrodynamic quantities are

$$\rho = 1, \quad (82)$$

$$v_x = 0.1 \sin(\pi y), \quad (83)$$

$$v_y = 0, \quad (84)$$

$$p = 0.1, \quad (85)$$

$$b_x = 0, \quad (86)$$

¹³ The setups for the ATHENA++ runs are available under <https://github.com/rainerweinberger/athena-public-version.git>, input files `athinput.tb` and `athinput.current_sheet`.

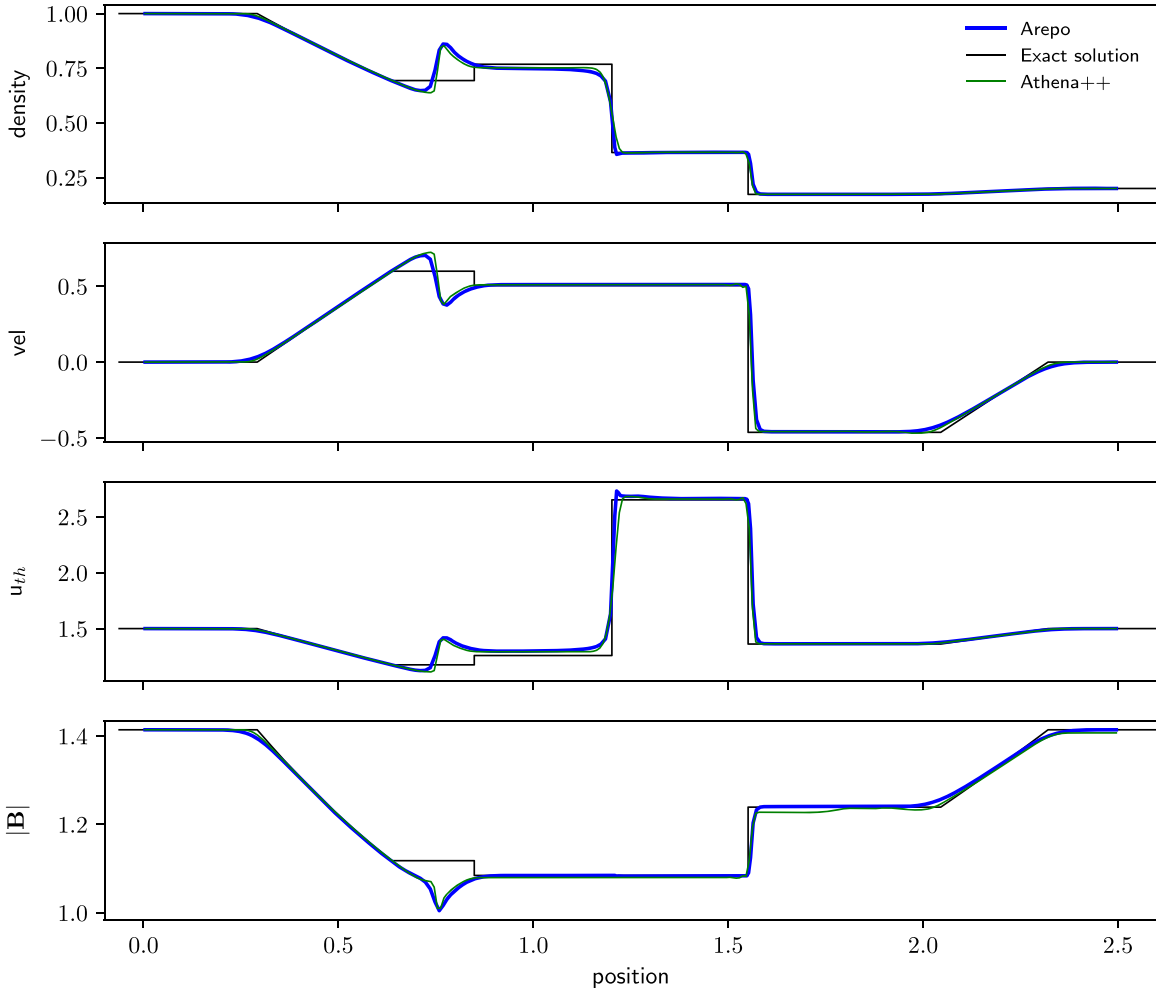


Figure 7. Magnetohydrodynamic properties of an MHD shocktube problem at time $t = 0.4$. The black line shows the exact solution, while the blue and green lines show solutions computed with AREPO and ATHENA++, respectively, using 300 cells.

$$b_y = \begin{cases} 1 & \text{if } x < 0.5 \text{ or } x > 1.5, \\ -1 & \text{else,} \end{cases} \quad (87)$$

and an adiabatic index $\gamma = 5/3$ is used.¹⁴ The cells are allowed to move, but refinement and de-refinement are disabled, and $C_{\text{CFL}} = 0.3$. To verify the solution, we check the total magnetic energy in the system. For comparison, we ran the same problem using the ATHENA++ code. Figure 10 shows slices of the magnetic energy density at $t = 1$ and the time evolution of the total magnetic energy in the simulation until $t = 30$, showing a higher numerical reconnection rate in the AREPO simulation.

10.7. Noh Shock Problem

As a test for the Riemann solver in two and three dimensions, we use Noh’s shock problem (Noh 1987) in a computational domain of side length $L = 6$, with open boundary conditions. Initial density, velocity, and pressure are given by

$$\rho = 1, \quad (88)$$

¹⁴ Note that the magnetic field here is given in Heaviside-Lorentz units, while the AREPO input is given in Gauss units.

$$\mathbf{v} = -\hat{\mathbf{r}}, \quad (89)$$

$$p = 10^{-4}, \quad (90)$$

where $\hat{\mathbf{r}}$ is the unit vector of the position relative to the domain center, i.e., we set up a constant velocity inflow to the center. The adiabatic index of the gas is $\gamma = 5/3$, and the Courant factor $C_{\text{CFL}} = 0.3$. The cells are allowed to move, without considering refinement or de-refinement. The inflowing material causes a strong shock to develop in the center, traveling outwards. The (analytically derivable) post-shock solution as well as the position of the shock are then checked in the verification (see Figure 11).

10.8. Gresho Vortex

In two dimensions, it is interesting to test the hydrodynamical scheme via a stationary vortex problem, i.e., a problem where the centrifugal forces are balanced by a pressure gradient. One such example is the Gresho vortex problem (Gresho & Chan 1990). We set up a two-dimensional box with side length $L = 1$ and periodic boundary conditions. For the density, azimuthal velocity, and pressure, we choose

$$\rho = 1, \quad (91)$$

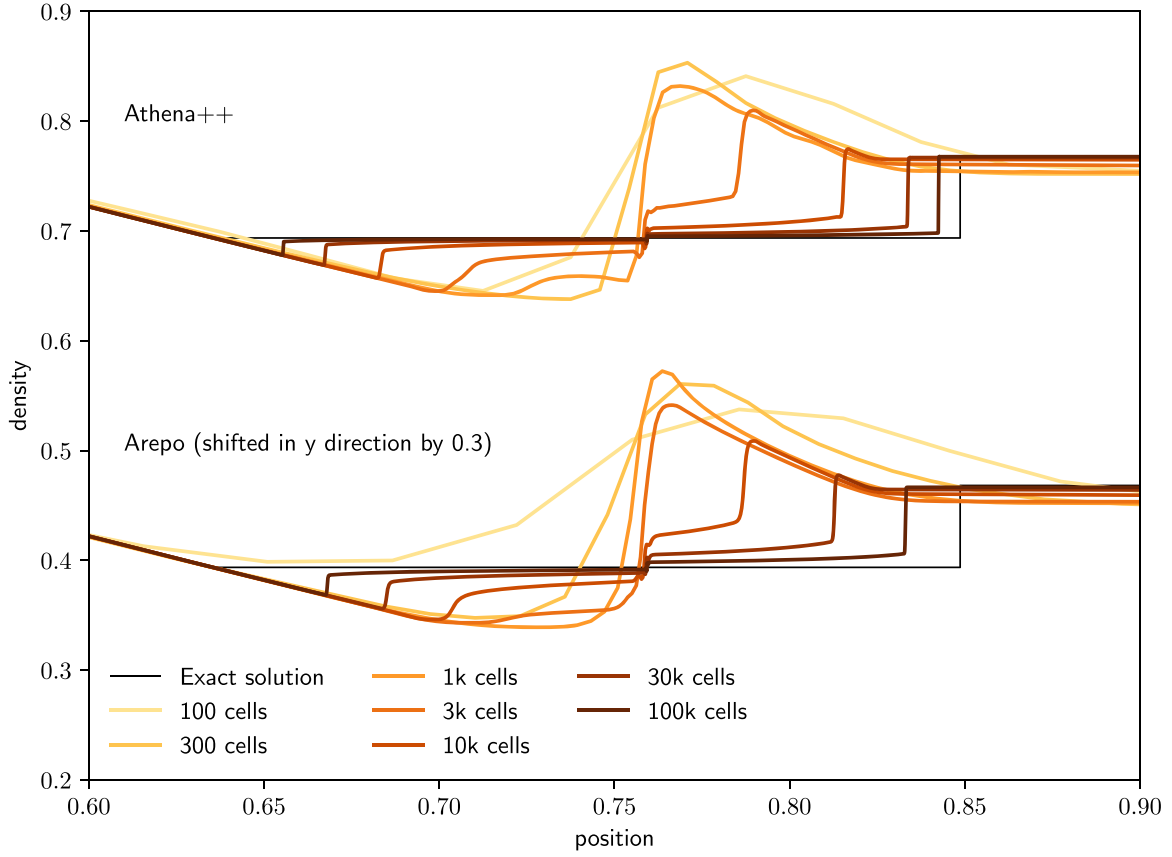


Figure 8. Density of an MHD shocktube problem at time $t = 0.4$ focused on positions between 0.6 and 0.9 for different resolutions. Both codes, AREPO and ATHENA++, converge toward the exact solution, but only at very high resolution. The convergence in the AREPO case is slightly slower, requiring a factor of three more cells to produce a comparable result.

$$v_\varphi = \begin{cases} 5r & \text{for } r < 0.2, \\ 2 - 5r & \text{for } 0.2 \leq r < 0.4, \\ 0 & \text{for } r \geq 0.4, \end{cases} \quad (92)$$

$$p = \begin{cases} 5 + 12.5r^2, & \text{for } r < 0.2, \\ 9 + 12.5r^2 - 20r + 4.0 \ln(r/0.2) & \text{for } 0.2 \leq r < 0.4, \\ 3 + 4 \ln(2) & \text{for } r \geq 0.4. \end{cases} \quad (93)$$

The initial radial velocity is zero. We use $C_{\text{CFL}} = 0.3$ and let the problem evolve with a moving mesh until $t = 3$, and verify the result by comparing to these initial conditions. We study the convergence using the density field in Figure 4, central panel.

10.9. Yee Vortex

The Yee vortex problem (Yee et al. 2000) is similar in nature to the Gresho vortex, however, has the advantage of having a smooth solution. We follow the setup of Pakmor et al. (2016b) and use a two-dimensional computational domain with side length $L = 10$ and periodic boundary conditions. Adopting a temperature at infinity of $T_{\text{inf}} = 1.0$, $\beta = 5.0$, and an adiabatic index $\gamma = 1.4$, the temperature profile is given by

$$T(r) = T_{\text{inf}} - \frac{(\gamma - 1)\beta^2}{8\pi^2\gamma} \exp(1 - r^2), \quad (94)$$

and the density ρ , azimuthal velocity v_φ , and specific internal energy u as a function of radius r are given by

$$\rho(r) = T(r)^{\frac{1}{\gamma-1}}, \quad (95)$$

$$v_\varphi(r) = \frac{r\beta}{2\pi} \exp\left(\frac{1 - r^2}{2}\right), \quad (96)$$

$$u(r) = \frac{T(r)}{\gamma - 1}. \quad (97)$$

The radial velocity is zero. We employ $C_{\text{CFL}} = 0.3$ and evolve this setup with a moving mesh until $t = 10$ and again verify the result by comparing with the initial conditions. We study the convergence using the density field in Figure 4, right panel.

10.10. Cosmological Simulation with Gravity Only

One of the most important types of simulations for cosmology is gravity-only cosmological volume simulations, one example being the Millennium simulation (Springel et al. 2005b). In our test case of this type of simulation, we employ a computational box of $50 h^{-1} \text{ Mpc}$ on a side and 32^3 simulation particles to serve as a computationally cheap example. This allows for a rapid calculation of the solution, which is then compared to a previously obtained one. In particular, the halo masses of the emerging structures are compared to a reference run that used the same initial conditions. The resulting structure

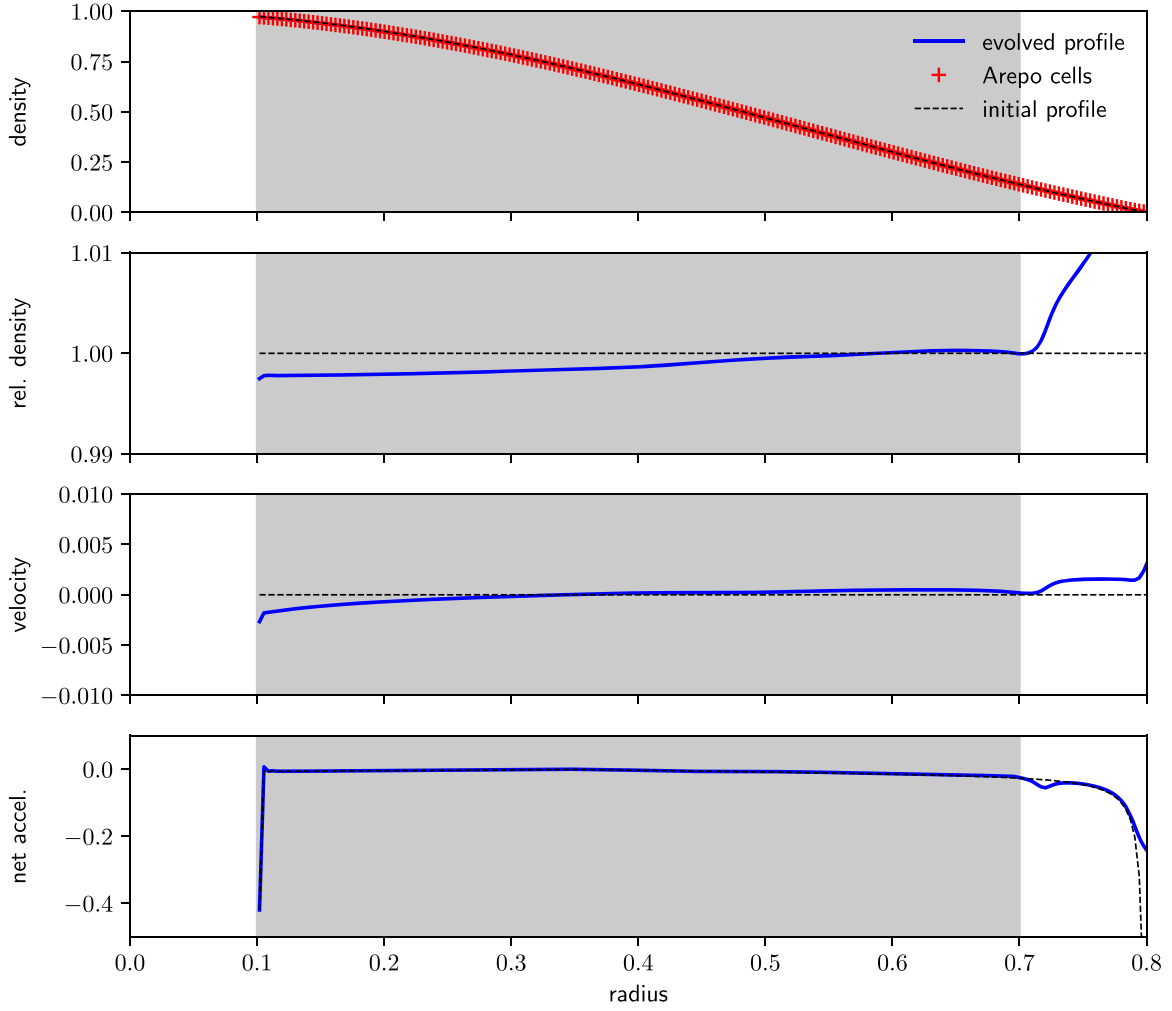


Figure 9. Density profile, relative density deviation from initial conditions, radial velocity, and net acceleration as a function of radius for a 1D spherically symmetric $n = 1$ polytrope setup.

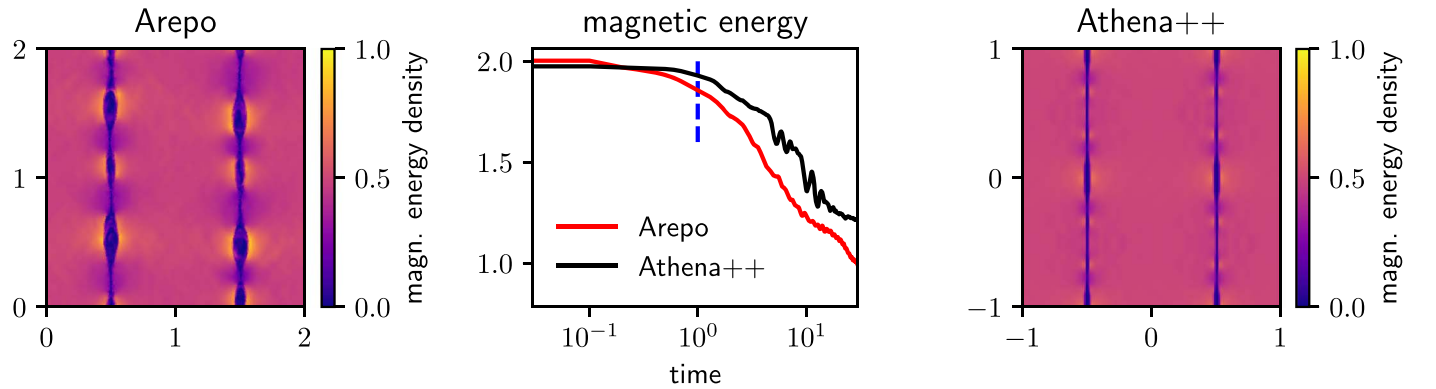


Figure 10. Left and right panels: magnetic energy density of the current-sheet problem at time $t = 1$ computed with AREPO and ATHENA++, respectively, each using 256^2 cells. The AREPO result shows a significantly growing instability driven by the larger numerical reconnection, while the ATHENA++ simulation remains relatively static to this point. Eventually, numerical reconnection will, in both cases, lead to a growth of the instability as can be seen in the time evolution of the magnetic energy in the simulation (central panel).

at redshift $z = 0$ is shown in Figure 12 with an inlay of the halo mass function. We note that the box is too small and the statistical power too limited to yield a meaningful comparison with fitting formulae for the halo mass function. We ensured, however, that the test is actually sensitive to the gravity calculation by varying

the gravitational constant by one percent. The resulting halo masses are then beyond the tolerance limit we impose on the test. The creation script for this test also easily allows for the creation of user-defined initial conditions via the N-GENIC and MUSIC (Hahn & Abel 2011) codes.

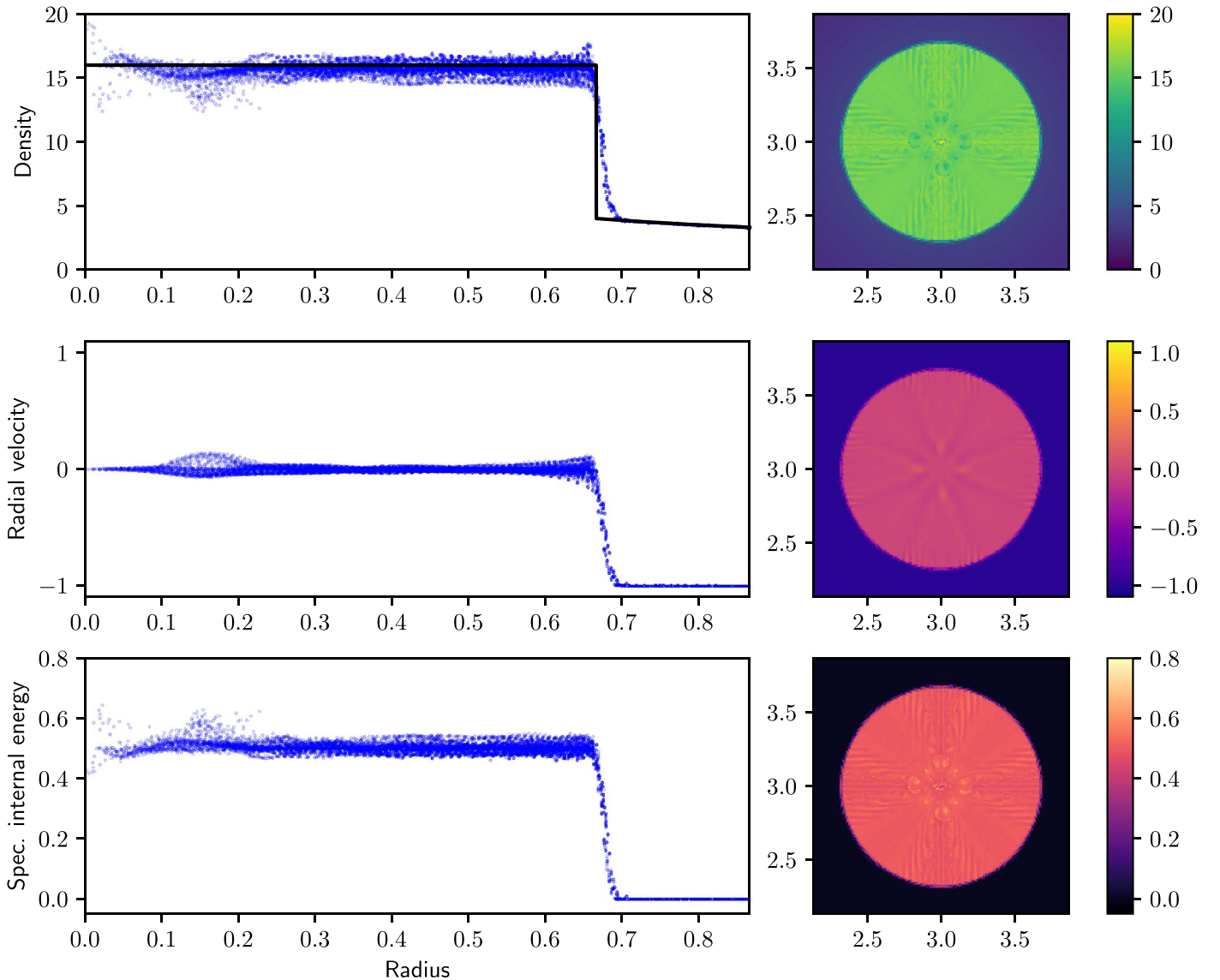


Figure 11. Two-dimensional Noh problem at time $t = 2$ using 150^2 initially uniformly spaced grid cells.

10.11. Cosmological Volume with Gas Cooling and Star Formation

Besides calculating gravitational interactions in cosmological volume simulations, it is also possible with AREPO to include gas in the initial conditions, and to allow for radiative cooling and the formation of stars in gas exceeding a density threshold. To resolve this, one needs a significantly higher mass resolution than is used in the previous test; thus, we employ a box of $7.5 h^{-1}$ Mpc on a side but still use only 32^3 collisionless particles.

AREPO reads these initial conditions and creates from them an additional gas component according to the specified cosmic baryon and matter density fractions. For this purpose, each dark matter particle is split into a gas cell and a dark matter particle of reduced mass, i.e., the simulation then has 2×32^3 resolution elements initially. Figure 13 shows that the stars (red) form at the centers of the dark matter (blue) overdensities. The setup used here is very similar to one of the cosmological simulations presented in Springel & Hernquist (2003), and in fact, the number of simulation resolution elements is

comparable. One of the key quantities we analyze is the star formation rate density as a function of redshift, on which the verification is based. As before, the solution is compared to a previous run, mainly because for this quantity the correct solution is not known. Note that the actual star formation rate in this particular test simulation will exceed the rates deduced from observations substantially, due to the absence of energetic feedback processes capable of driving galactic outflows (see Springel & Hernquist 2003). As in the previous example, the creation script for this test also allows for the creation of user-defined initial conditions via the N-GENIC and MUSIC codes.

10.12. Cosmological Zoom Simulation with Gravity Only

Another important type of simulation often used in galaxy formation studies is cosmological “zoom” simulations. These are set up in a similar way as the cosmological runs discussed earlier; however, they employ a nonuniform mass resolution. In particular, for constructing such zooms, a Lagrangian region for an object of interest is identified as the region where all of the particles that end up in the object at the final snapshot time

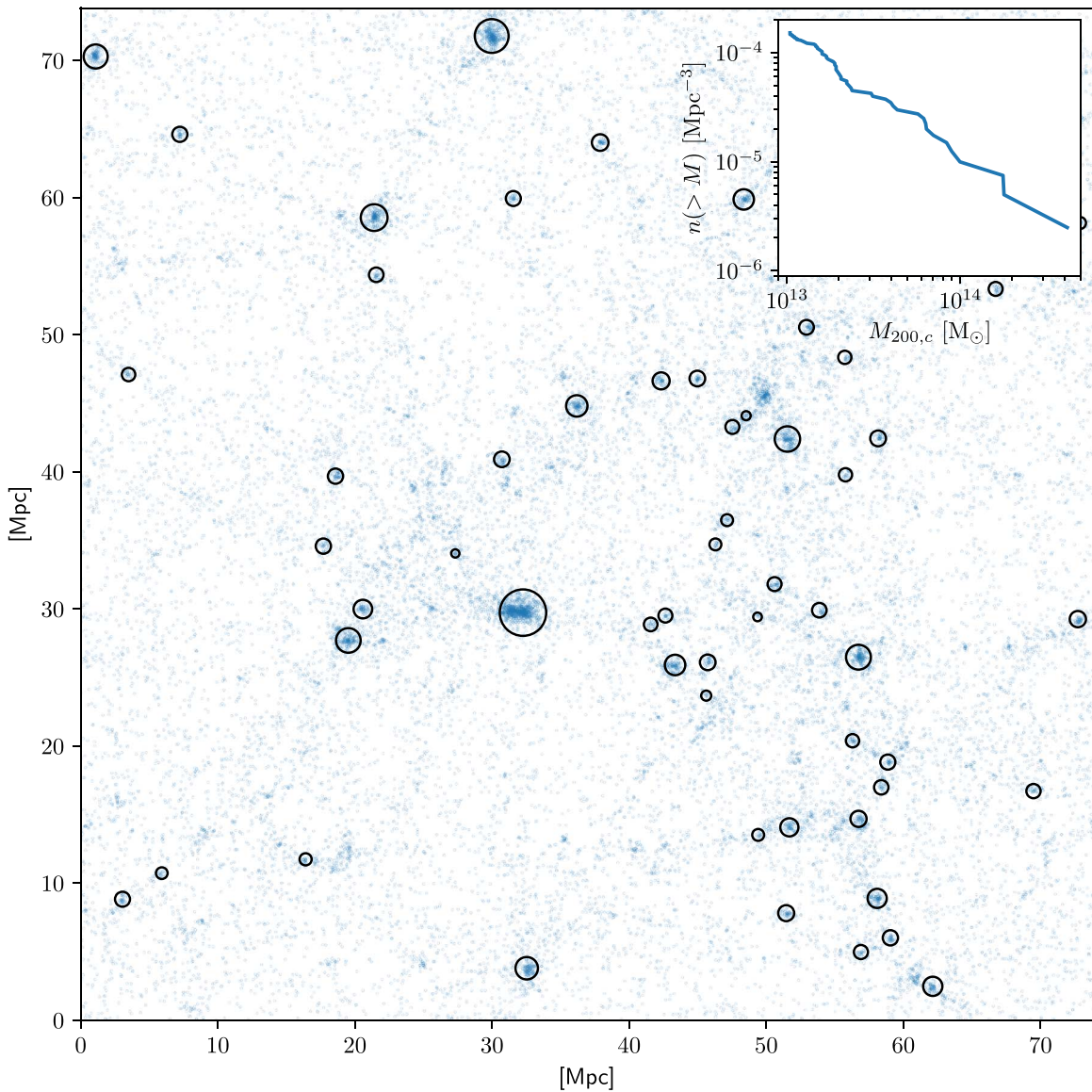


Figure 12. The cosmic large-scale structure in a gravity-only cosmological simulation. The blue dots denote the simulation particles of a very low-resolution (32^3) simulation, the black circles denote the halos identified by the SUBFIND algorithm, with sizes indicating their corresponding virial radii $R_{200, c}$. The inlay shows the halo mass function in the simulation volume.

originated from. This region is then significantly refined in the initial conditions, whereas the rest of the cosmological volume is sampled with poorer resolution. This allows the large-scale tidal field originating from these regions to still be captured without having to track its evolution in highly resolved detail. In this way, it is possible to simulate individual objects in their full cosmological context and afford significantly better mass resolution for them. AREPO has a number of features specifically tuned for these kinds of simulations, in particular, allowing the high clustering of the computational workload to still balance well.

We thus include a test of a zoom run to verify this functionality. Figure 14 shows the redshift $z = 0$ particle distribution and the identified subhalos of the example halo, as well as the subhalo mass function in the inlay. As a primary diagnostic of the test, we compare the subhalo masses with a precomputed reference run with the same initial conditions as the simulation itself. The creation script in this case relies on the MUSIC code.

10.13. Isolated Object, Galaxy Gravity Only

For studying the dynamics of an individual galaxy, a slightly different kind of setup can be used. In particular, to study galactic dynamics problems, one might be interested in setting up a stationary collisionless galaxy and studying its evolution. Setting up such a galaxy in equilibrium is a nontrivial task. Here we use the GALIC code (Yurin & Springel 2014) to create initial conditions of a compound galaxy model, based on their model D, which has a spherically symmetric dark matter halo and an exponential stellar disk. Since this is a stationary problem, it is straightforward to verify the simulation by looking for the absence of secular trends. We here measure the vertical scale height of the disk and its evolution in time.

10.14. Galaxy Merger, Including Gas and Star Formation

A very informative numerical experiment to study galaxy transformations consists of two isolated galaxies that are put on a collision course. Here we follow the setup used by Springel &

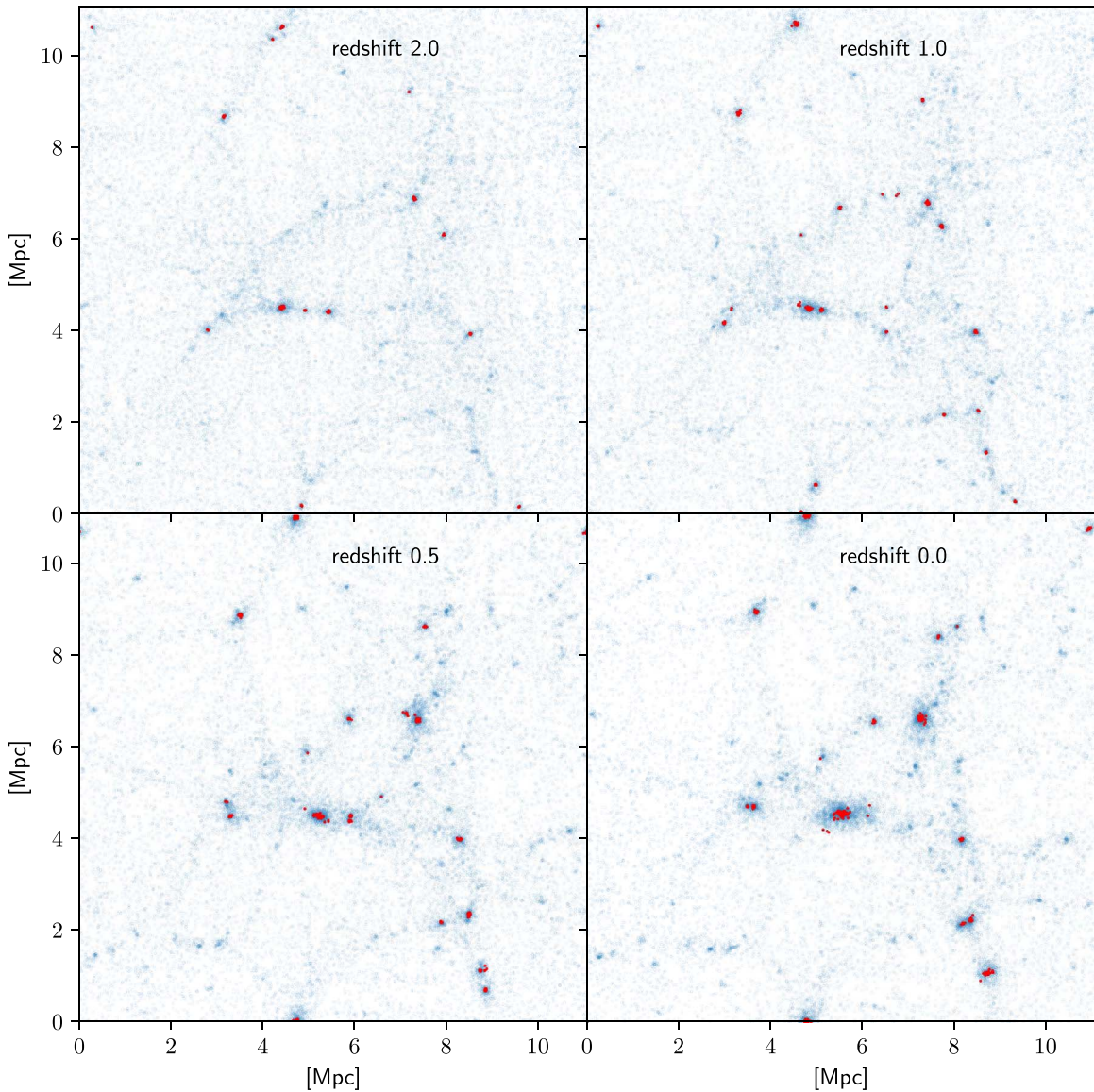


Figure 13. Cosmic dark matter (blue) and stellar (red) distributions at different redshifts for a cosmological volume simulation including gas cooling and star formation.

White (1999) and Springel et al. (2005a) to study the merger of two disk galaxies on a parabolic encounter orbit. Each galaxy includes a dark matter halo, pre-existing stars, and gas. We note that the initial conditions we use were designed originally for the SPH codes GADGET and GADGET-2 (Springel et al. 2001; Springel 2005), which implies a number of small but important differences in how the gas in the initial conditions is represented.

The most important difference is the difficulty of grid-based codes to deal with a complete vacuum. While it is straightforwardly possible in SPH codes to simply not place any gas particles into the background, a grid code typically requires a nonvanishing gas density in these regions, because otherwise, pesky stability issues arise due to tiny floating point values and associated round-off errors. As was discussed earlier, we therefore use the special `ADDBACKGROUNDGRID` mode of AREPO to convert the SPH initial conditions to ones where the density field is defined everywhere on a grid. These are then evolved by AREPO subsequently. Figure 15 shows a time series of the star particles in this simulation with the trajectories of the galaxies indicated by the blue dotted and

dashed lines. For verification, the resulting star formation rates are compared against a reference solution.

10.15. General Remarks on Initial Condition Generation

The most important characteristic feature of AREPO compared to Cartesian mesh codes or smoothed particle hydrodynamics codes is that the choice of grid coordinates can be quite arbitrary and may evolve in time in a smooth fashion. This allows a flexible adjustment of the geometry of resolution elements and their local resolution to the problem at hand.

As an example, in approximately spherically symmetric configurations, such as in 3D hydrodynamic simulation of a star, the mesh can be set up in a spherically symmetric fashion with radially adaptive cell volumes, keeping the mass per cell approximately equal (Ohlmann et al. 2017). This flexibility in positioning the mesh can, in principle, allow a simulation to have fewer cells at equal resolutions than are possible with Cartesian adaptive mesh refinement techniques, which are more

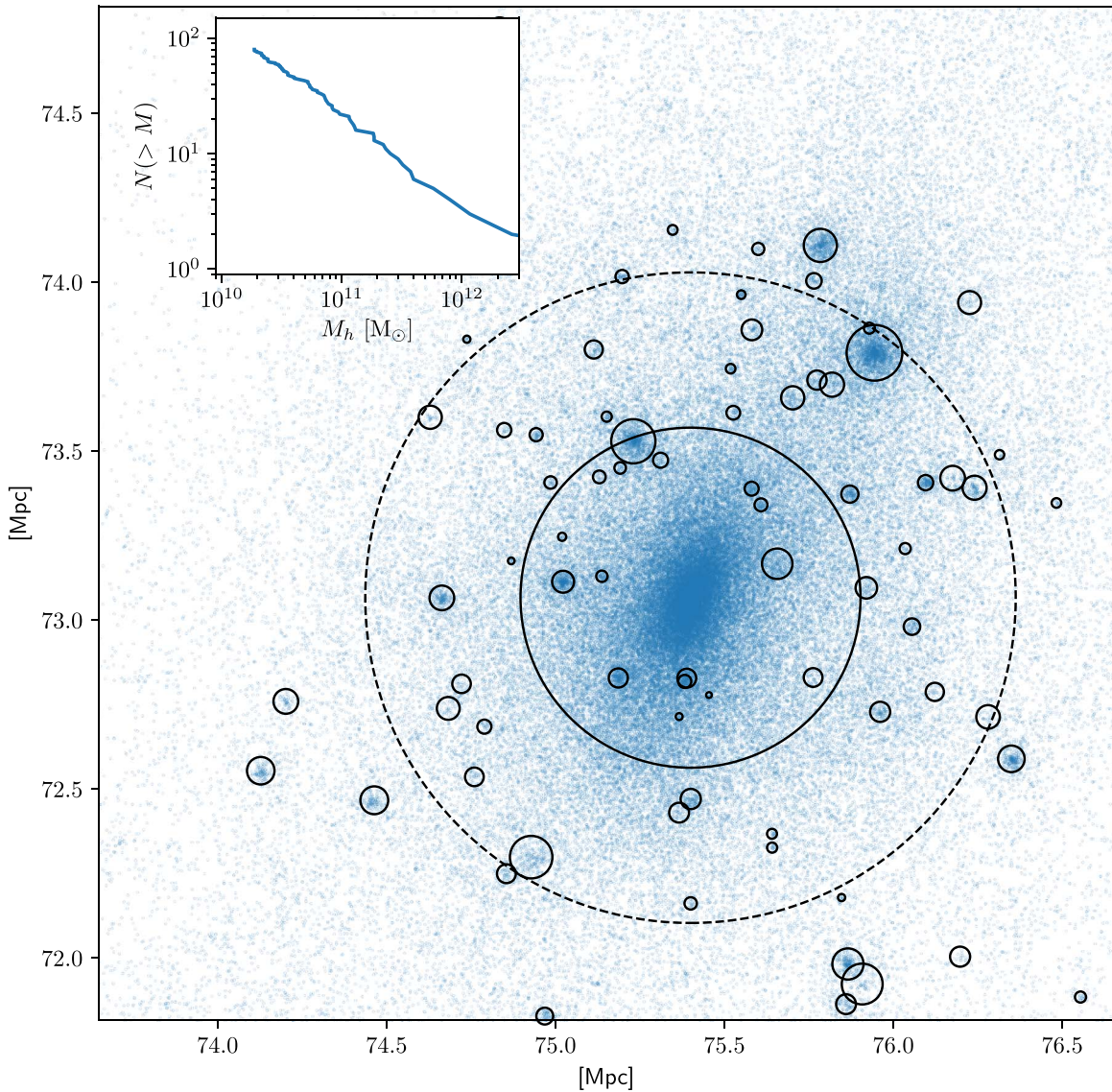


Figure 14. Halo structure of a zoom simulation of a $\sim 10^{14} M_{\odot}$ dark matter halo. The black solid circles denote the subhalos and their half mass radii as determined by the SUBFIND algorithm. The dashed circle denotes $R_{200,c}$ of the main halo. The inset shows the subhalo mass function within $2 R_{200,c}$ of the main halo.

limited in their mesh geometries. Yet, there are a number of considerations that need to be observed to arrive at high-quality initial conditions.

AREPO reads in positions of mesh-generating points and creates the gas cells from them using a Voronoi tessellation for these points. The mesh creation algorithm is described in detail in Springel (2010b) and relies on a number of geometric predicates that test whether points lie inside or outside of circum-circles around tetrahedra (or triangles in 2D). The code here relies on an unambiguous and correct answer to these geometric tests. This poses additional challenges due to numerical floating point round-off when degenerate mesh configurations are encountered, for example, when more than four points lie exactly on the same sphere. One case where this occurs is a Cartesian configuration of the mesh-generating points.

While AREPO is able to cope with such situations, it needs to employ exact arithmetic to robustly cope with the degeneracies in such situations, slowing down the mesh construction. In practice, this implies that starting out with a perfect Cartesian

mesh leads to a considerable amount of time spent in the first couple of mesh constructions, until the mesh-generating points have started to slowly dissolve the Cartesian configuration due to the fluid motion. To avoid this, one may use slight displacements from an ideal Cartesian mesh, or better yet, use a honeycomb-like configuration. Note that it is not necessary to supply the mass of each cell in the initial conditions; one can instead also supply the density of each cell. This greatly simplifies the placement of initial mesh-generating points, as one does not need to know the resulting cell volumes of the Voronoi tessellation ahead of time. Instead, one can simply supply the desired primitive variables for each cell in the initial conditions, and the code then computes the conserved quantities after the first mesh has been constructed and the cell volumes are known.

For convergence tests or setups that require high-quality initial conditions, an additional complication has to be considered: in a finite-volume-based scheme, the primitive variables represent average values of the cells. Using the value of a desired input at the center of mass is usually a good

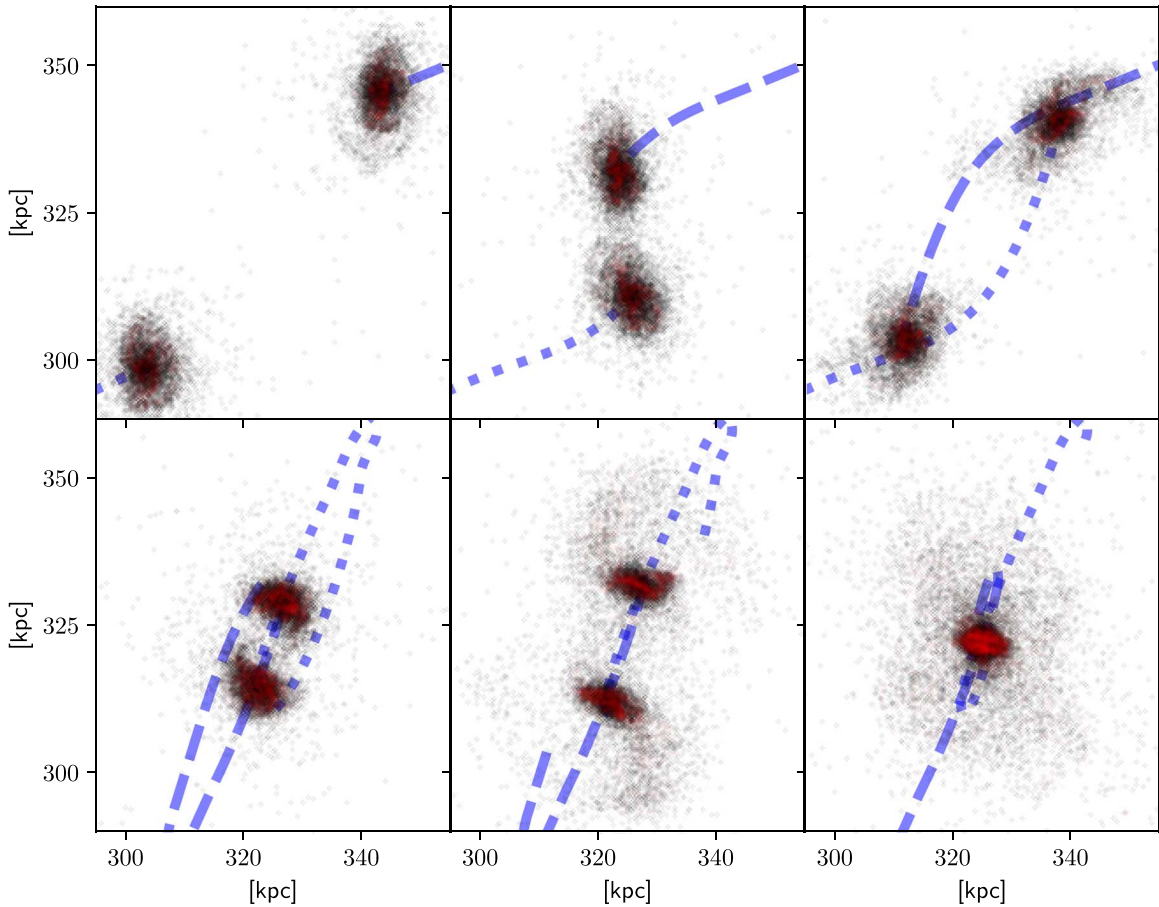


Figure 15. Time series of the position of existing (black) and newly formed (red) stars in a galaxy merger of two equal-mass galaxies. The blue lines indicate the trajectory of the center of mass of each of the galaxies. The upper row shows the first pericenter passage, and the lower row shows the second pericenter passage and the final merger.

enough approximation for each cell; however, for a Voronoi cell, the position of the mesh-generating point (which is specified in the initial conditions as the position) is in general *not* the center of mass. Thus, using the primitive variables at the mesh-generating point can degrade the quality of the initial conditions, depending on the used cell shapes.

11. Code Development Aspects

The AREPO code as presented here has in total almost 10^5 lines of code, making it about a factor of five larger than the GADGET-2 code (Springel et al. 2005b). The main contribution to the increased complexity comes from the added algorithms for mesh construction and hydrodynamics, but also other numerical parts have become more sophisticated. However, we stress that the code still only gives basic functionality for astrophysical simulations, despite being already of quite large size. In many research applications, additional features such as, e.g., more sophisticated radiative cooling routines, a tracking of different chemical elements, nuclear reaction networks, treatments of radiative transfer, or more sophisticated sub-grid models are highly desirable or essential. Including such modules can easily increase the size of the code by another factor of several. The corresponding work exceeds the capacity of individual researchers, but becomes possible by a community effort. We therefore would like to actively encourage contributions by external developers in the form of code modules and extensions developed independently from the

original authors. In the following, we discuss how we imagine such contributions can be made based on the publicly released version of the code, and how this relates to other versions of the code.

11.1. The AREPO Development Version

The original AREPO code grew considerably over the years both in scope and number of developers/users. Currently, more than 150 scientists have been given access to the developer version (for comparison, in 2014 October, this number was 50). The source code exceeds 3×10^5 lines (less than 2×10^5 lines in 2014 October). This growth in size, and perhaps more importantly in the number of developers and users, has required us in recent years to adopt a more formalized development workflow than the anarchic model followed by GADGET and AREPO in the past.

The development version of AREPO is now organized around a master branch, in which all working, well-tested modules are included. For the development of a specific feature, a dedicated branch is opened. This branch is ideally kept in sync with the master branch, and merged back to master once the feature is ready for production level use. For the integration within the master branch, the approval of one of the senior developers is required. For major simulation projects, it has also proven practical to create a separate branch with a stable code version dedicated to this project, and which is then only updated with bug fixes.

Having all production level features included in the master branch allows the main developers to make changes to foundations of the code basis, such as, e.g., the MPI communication layer, and still ensure functionality of all aspects of the code afterwards. However, one of the disadvantages of this workflow is that the main code is prone to become very crowded and complex eventually, and guidelines with respect to code style and modularity need to be enforced as well. Since the level of expertise in scientific code development among the users and developers varies vastly, with most people in computational astrophysics not having much experience with large software development projects, this complexity can become a severe obstacle for new developers to get started. It also requires substantial amounts of time (proportional to the number of developers) from senior developers to maintain the code. We therefore concluded that this full-fledged development model is not well suited for the public version of AREPO.

11.2. Development Philosophy of the Public Version

Instead, we decided to provide a stable base version of the code, as presented in this paper, which will be made available through a powerful version control system (GIT in this case), but is not intended for significant further development apart from bug fixes and possibly the addition of further examples. Developments of new features and extensions should be made in separate branches, forks, or user's own repositories instead. We further encourage developers to provide access to their branch and/or their model as a patch to the (stable) master branch, such that interested users can apply or include it when needed. In this way, each model development is independent of each other, and does not need involvement of the original AREPO authors. However, we encourage developers of new models to contact us when they want to release their patches publicly, so that we can maintain a list of available modules. The authors are also happy to host stable versions as model branches next to the main repository. It is obviously important in this approach that model developments are based on a proper and consistent use of version control software tools, so that the simple creation of patches is possible, and so that updates of the master branch, in case they happen, can easily be merged into them, too.

11.3. Recommendations for Additional Models

We note that the AREPO code still shares many resemblances in its internal code structure with the GADGET-2 and GADGET-3 codes. This will allow everyone who is familiar with either of these codes to have an easy start with AREPO, and makes it relatively straightforward to port modules written for any version of GADGET to AREPO.

When turning such a port into a module patch for the public version, it is highly advisable to consider a number of aspects. While the base of the code is planned to be relatively static, there might be a large number of different modules from other users, and some simulations might require multiple modules. It is therefore highly advisable to design a module in such a way that the changes to the original code are reduced to a small number of unavoidable function-calls, whereas all of the extra functionality is implemented in new source code files. This way of coding reduces the number of potential conflicts between different modules significantly, and will, in many cases, allow their joint use with no or little extra work. Additionally, we

encourage developers to include (small) model tests of their own as an example of the added functionality, and to allow easy verification of the new module when other changes are applied as well. While it is not possible to guarantee correct functionality in all cases, the consistent use of such examples is by experience extremely helpful for detecting usage problems, bugs, or model incompatibilities early on.

11.4. Bug Reporting and User Support

The continuing trend toward more complex astrophysical simulation software makes training new users on how to use and improve the software ever more important. The main effort in making this public release possible was in fact providing a complete documentation of the source code and of various details of its usage, as well as preparing small, out-of-the-box working examples as a guideline to set up new simulations.

However, even the most complete user guide and the most extensive set of examples will not cover all possible issues that arise. In the past, remaining questions could be clarified by contacting the author directly, e.g., via email. This is, in principle, still possible, but it has significant disadvantages. The solution to a specific problem arising from such an interaction is only available to the person that contacted the author, and may not be archived/accessible in a systematic way so that many people may run into the same problem time and again. Also, answering all requests may require a significant amount of time, which eventually decreases the quality of the support that a given number of people can provide.

To improve the level of support, we host a support forum on the code's website,¹⁵ where users can report their problems, and, importantly, also answer the problems of other users. This way, we hope to create a supportive user community, in which more experienced users provide help to less experienced ones. If successful, this will allow the user base to grow without compromising the level of support that a new user can get, as well as producing a public knowledge base for questions related to the code's use. Bug reporting from users will also happen via this forum, while the issue-tracking system of the code will serve as listing confirmed bugs. Users will be notified about code updates via a blog on the code's website.

12. Summary and Conclusions

We introduced the public open-source version of the AREPO code and its underlying algorithms. AREPO calculates the time evolution of initial-value problems of MHD using a finite-volume approach on a moving unstructured Voronoi mesh. Other open-source codes that adopted a similar approach include RICH (Yalinewich et al. 2015) and SHADOWFAX (Vandenbroucke & de Rijcke 2016). Additionally, AREPO calculates gravitational forces using a tree-particle-mesh technique, making it possible to include multiple species of collisionless (i.e., only gravitationally interacting) particles.

Time integration is performed for each element with its individual local time step criterion, allowing for efficient calculations also in situations with a large dynamic range in time. Additional gas physics such as radiative cooling and a very simple model for the unresolved interstellar medium are included as well, primarily serving as guidelines for how such extra physics can be coupled to the main code. A built-in FOF

¹⁵ www.arepo-code.org/forums

group finder and the substructure identification algorithm SUBFIND, which may be used on-the-fly or in post-processing, as well as routines to convert smoothed particle hydrodynamics initial conditions to the appropriate equivalent AREPO-input are further parts of the code base and should simplify a productive use of the code especially for simulations of cosmic structure formation.

With this public release, we hope to provide the community with a useful tool for future research in theoretical astrophysics, and with a good basis for further code development based on the concept of a moving mesh. The source code is completely documented, and examples of different complexity are provided to facilitate getting started with the code. We also provide a support forum that is hosted on the code's website as a platform to ask questions about the code, and to report potential bugs if present. Ideally, this will grow into an active and supportive user community that enriches the work of young students and senior computational astrophysicists alike. Scientific developers are encouraged to extend the code base in their own repositories, and provide their modules via patches or public branches to the community. While this can, in principle, be done completely independently of the authors, we are also happy to host well-tested branches as separate module branches and list them on the code's website. It remains to be seen how well the particular model envisaged here works in practice. We are of course open to make adjustments if they are indicated.

There is certainly no shortage of ideas for the next development steps to improve the performance and capabilities of AREPO, some of which have already been started. Higher-order methods, both in MHD (Schaal et al. 2015; Guillet et al. 2019) and gravity (V. Springel et al. 2020, in preparation) are very interesting for improving accuracy for a given computational expense. Also, AREPO has started to include further physical effects such as radiation (Petkova & Springel 2011; Jaura et al. 2018; Kannan et al. 2019), cosmic rays (Pakmor et al. 2016a; Pfrommer et al. 2017), as well as nonideal hydrodynamics and plasma-physics effects (Kannan et al. 2016; Marinacci et al. 2018a; T. Berlok et al. 2020, in preparation). Significant further research and additional development needs to be done to improve the accuracy and universal applicability of these modules.

Other challenges lie in the distributed-memory parallelization layer, which needs to be improved such that scalability can be extended to much larger machine sizes. This is especially difficult for very aggressive zoom simulations with their highly nonuniform resolution and extreme dynamic range in density and timescales. Similarly, special-purpose hardware such as graphics processing units may need to be embraced on the basis of redesigned algorithms that better map to their streaming processors. Another, sometimes overlooked challenge is to minimize the energy used for the solution. Optimizing this may well become very important for the use of future supercomputers.

Continuous improvement of numerical methods and codes is clearly a prerequisite to ensure progress in computational astrophysics and to allow the technical advances in computer performance to be turned into answers to open questions in theoretical astrophysics. We are convinced that the public release of AREPO, joining the general trend for open, well-documented scientific software and following the example of other codes, is a step in the right direction, helping to support

reproducibility and scientific progress with computational methods in the light of ever more complex simulation software.

The authors would like to thank the full user base of AREPO for their continued encouragement to realize a public release of the code, and for their long-standing efforts in putting the code to great scientific use as well as the anonymous referee for the excellent suggestions that greatly improved the quality of this manuscript. R.W., V.S., and R.P. would like to thank financial support by the Priority Programme 1648 “SPPEXA” of the German Science Foundation, and by the European Research Council through ERC-StG grant EXAGAL-308037.

ORCID iDs

Rainer Weinberger  <https://orcid.org/0000-0001-6260-9709>
Volker Springel  <https://orcid.org/0000-0001-5976-4599>

References

- Bagla, J. S. 2002, *JApA*, **23**, 185
Barnes, J., & Hut, P. 1986, *Natur*, **324**, 446
Bauer, A., & Springel, V. 2012, *MNRAS*, **423**, 2558
Bourne, M. A., & Sijacki, D. 2017, *MNRAS*, **472**, 4707
Bryan, G. L., Norman, M. L., O'Shea, B. W., et al. 2014, *ApJS*, **211**, 19
Cen, R. 1992, *ApJS*, **78**, 341
Duffell, P. C., & MacFadyen, A. I. 2011, *ApJS*, **197**, 15
Evans, C. R., & Hawley, J. F. 1988, *ApJ*, **332**, 659
Felker, K. G., & Stone, J. M. 2018, *JCoPh*, **375**, 1365
Fiacconi, D., Sijacki, D., & Pringle, J. E. 2018, *MNRAS*, **477**, 3807
Fromang, S., Hennebelle, P., & Teyssier, R. 2006, *A&A*, **457**, 371
Gardiner, T. A., & Stone, J. M. 2005, *JCoPh*, **205**, 509
Genel, S., Vogelsberger, M., Springel, V., et al. 2014, *MNRAS*, **445**, 175
Gnedin, N. Y., Semenov, V. A., & Kravtsov, A. V. 2018, *JCoPh*, **359**, 93
Goicovic, F. G., Springel, V., Ohlmann, S. T., & Pakmor, R. 2019, *MNRAS*, **487**, 981
Grand, R. J. J., Gómez, F. A., Marinacci, F., et al. 2017, *MNRAS*, **467**, 179
Gresho, P. M., & Chan, S. T. 1990, *IJNMF*, **11**, 621
Guillet, T., Pakmor, R., Springel, V., Chandrasekar, P., & Klingenberg, C. 2019, *MNRAS*, **485**, 4209
Hahn, O., & Abel, T. 2011, *MNRAS*, **415**, 2101
Hernquist, L., Bouchet, F. R., & Suto, Y. 1991, *ApJS*, **75**, 231
Hockney, R. W., & Eastwood, J. W. 1981, *Computer Simulation Using Particles* (New York: McGraw-Hill)
Jacob, S., Pakmor, R., Simpson, C. M., Springel, V., & Pfrommer, C. 2018, *MNRAS*, **475**, 570
Jaura, O., Glover, S. C. O., Klessen, R. S., & Paardekooper, J. P. 2018, *MNRAS*, **475**, 2822
Jiang, Y.-F., Belyaev, M., Goodman, J., & Stone, J. M. 2013, *NewA*, **19**, 48
Kannan, R., Springel, V., Pakmor, R., Marinacci, F., & Vogelsberger, M. 2016, *MNRAS*, **458**, 410
Kannan, R., Vogelsberger, M., Marinacci, F., et al. 2019, *MNRAS*, **485**, 117
Katz, N., Weinberg, D. H., & Hernquist, L. 1996, *ApJS*, **105**, 19
Lehner, L., & Pretorius, F. 2014, *ARA&A*, **52**, 661
Marinacci, F., Vogelsberger, M., Kannan, R., et al. 2018a, *MNRAS*, **476**, 2476
Marinacci, F., Vogelsberger, M., Pakmor, R., et al. 2018b, *MNRAS*, **480**, 5113
Menon, H., Wesolowski, L., Zheng, G., et al. 2015, *ComAC*, **2**, 1
Mignone, A., & Tzeferacos, P. 2010, *JCoPh*, **229**, 2117
Mo, H., van den Bosch, F. C., & White, S. 2010, *Galaxy Formation and Evolution* (Cambridge: Cambridge Univ. Press)
Mocz, P., Burkhardt, B., Hernquist, L., McKee, C. F., & Springel, V. 2017, *ApJ*, **838**, 40
Muñoz, D. J., Kratter, K., Springel, V., & Hernquist, L. 2014, *MNRAS*, **445**, 3475
Naab, T., Klessen, R. S., Kley, W., & Röpke, F. 2017, in *Proc. Perspectives of Astrophysics in Germany 2017-2030 Conf.*, ed. M. Steinmetz et al. (Potsdam: RDS), 18
Naiman, J. P., Pillepich, A., Springel, V., et al. 2018, *MNRAS*, **477**, 1206
Nelson, D., Pillepich, A., Genel, S., et al. 2015, *A&C*, **13**, 12
Nelson, D., Pillepich, A., Springel, V., et al. 2018, *MNRAS*, **475**, 624
Nelson, D., Pillepich, A., Springel, V., et al. 2019a, *MNRAS*, **490**, 3234
Nelson, D., Springel, V., Pillepich, A., et al. 2019b, *ComAC*, **6**, 2
Noh, W. F. 1987, *JCoPh*, **72**, 78

- Ohlmann, S. T., Röpke, F. K., Pakmor, R., & Springel, V. 2016, *ApJL*, **816**, L9
- Ohlmann, S. T., Röpke, F. K., Pakmor, R., & Springel, V. 2017, *A&A*, **599**, A5
- Pakmor, R., Kromer, M., Taubenberger, S., & Springel, V. 2013, *ApJL*, **770**, L8
- Pakmor, R., Pfrommer, C., Simpson, C. M., Kannan, R., & Springel, V. 2016a, *MNRAS*, **462**, 2603
- Pakmor, R., & Springel, V. 2013, *MNRAS*, **432**, 176
- Pakmor, R., Springel, V., Bauer, A., et al. 2016b, *MNRAS*, **455**, 1134
- Pelupessy, F. I., Jänes, J., & Portegies Zwart, S. 2012, *NewA*, **17**, 711
- Petkova, M., & Springel, V. 2011, *MNRAS*, **415**, 3731
- Pfrommer, C., Pakmor, R., Schaal, K., Simpson, C. M., & Springel, V. 2017, *MNRAS*, **465**, 4500
- Pillepich, A., Nelson, D., Hernquist, L., et al. 2018, *MNRAS*, **475**, 648
- Pillepich, A., Nelson, D., Springel, V., et al. 2019, *MNRAS*, **490**, 3196
- Portegies Zwart, S. 2018, *Sci*, **361**, 979
- Powell, K. G., Roe, P. L., Linde, T. J., Gombosi, T. I., & de Zeeuw, D. L. 1999, *JCoPh*, **154**, 284
- Power, C., Navarro, J. F., Jenkins, A., et al. 2003, *MNRAS*, **338**, 14
- Price, D. J., Wurster, J., Tricco, T. S., et al. 2018, *PASA*, **35**, e031
- Salmon, J. K., Winckelmans, G. S., & Warren, M. S. 1994, *IJSap*, **8**, 129
- Schaal, K., Bauer, A., Chandrashekar, P., et al. 2015, *MNRAS*, **453**, 4278
- Simpson, C. M., Pakmor, R., Marinacci, F., et al. 2016, *ApJL*, **827**, L29
- Smith, R. J., Glover, S. C. O., Clark, P. C., Klessen, R. S., & Springel, V. 2014, *MNRAS*, **441**, 1628
- Sod, G. A. 1978, *JCoPh*, **27**, 1
- Sparre, M., Pfrommer, C., & Vogelsberger, M. 2019, *MNRAS*, **482**, 5401
- Springel, V. 2005, *MNRAS*, **364**, 1105
- Springel, V. 2010a, *ARA&A*, **48**, 391
- Springel, V. 2010b, *MNRAS*, **401**, 791
- Springel, V., di Matteo, T., & Hernquist, L. 2005a, *MNRAS*, **361**, 776
- Springel, V., & Hernquist, L. 2003, *MNRAS*, **339**, 289
- Springel, V., Pakmor, R., Pillepich, A., et al. 2018, *MNRAS*, **475**, 676
- Springel, V., & White, S. D. M. 1999, *MNRAS*, **307**, 162
- Springel, V., White, S. D. M., Jenkins, A., et al. 2005b, *Natur*, **435**, 629
- Springel, V., Yoshida, N., & White, S. D. M. 2001, *NewA*, **6**, 79
- Stone, J. M., Gardiner, T. A., Teuben, P., Hawley, J. F., & Simon, J. B. 2008, *ApJS*, **178**, 137
- Teyssier, R. 2002, *A&A*, **385**, 337
- Teyssier, R. 2015, *ARA&A*, **53**, 325
- Toro, E. 1997, *Riemann Solvers and Numerical Methods for Fluid Dynamics* (Berlin: Springer), doi:10.1007/978-3-662-03490-3
- Torrilhon, M., & Balsara, D. S. 2004, *JCoPh*, **201**, 586
- Tóth, G. 2000, *JCoPh*, **161**, 605
- van de Voort, F., Springel, V., Mandelker, N., van den Bosch, F. C., & Pakmor, R. 2019, *MNRAS*, **482**, L85
- Vandenbroucke, B., & de Rijcke, S. 2016, *A&C*, **16**, 109
- Vogelsberger, M., Genel, S., Springel, V., et al. 2014, *MNRAS*, **444**, 1518
- Vogelsberger, M., Marinacci, F., Torrey, P., & Puchwein, E. 2020, *NatRP*, **2**, 42
- Vogelsberger, M., Sijacki, D., Kereš, D., Springel, V., & Hernquist, L. 2012, *MNRAS*, **425**, 3024
- Weinberger, R., Ehlert, K., Pfrommer, C., Pakmor, R., & Springel, V. 2017, *MNRAS*, **470**, 4530
- White, C. J., Stone, J. M., & Gammie, C. F. 2016, *ApJS*, **225**, 22
- Yalinewich, A., Steinberg, E., & Sari, R. 2015, *ApJS*, **216**, 35
- Yee, H. C., Vinokur, M., & Djomehri, M. J. 2000, *JCoPh*, **162**, 33
- Yoshikawa, K., Yoshida, N., & Umemura, M. 2013, *ApJ*, **762**, 116
- Yurin, D., & Springel, V. 2014, *MNRAS*, **444**, 62